

第28章 LCD—液晶显示

本章参考资料：《STM32F4XX-中文参考手册》、《STM32F4xx 数据手册》、库帮助文档《stm32f4xx_dsp_stdperiph_lib_um.chm》。

关于开发板配套的液晶屏控制器参数可查阅《ILI9806G-Data Sheet.pdf》资料获知。

本章讲解的内容涉及对 FSMC 的控制，若您不了解 FSMC 外设，请先学习前面的《FSMC—扩展外部 SRAM》章节。

28.1 显示器简介

显示器属于计算机的 I/O 设备，即输入输出设备。它是一种将特定电子信息输出到屏幕上再反射到人眼的显示工具。常见的有 CRT 显示器、液晶显示器、LED 点阵显示器及 OLED 显示器。

28.1.1 液晶显示器

液晶显示器，简称 LCD(Liquid Crystal Display)，相对于上一代 CRT 显示器(阴极射线管显示器)，LCD 显示器具有功耗低、体积小、承载的信息量大及不伤眼的优点，因而它成为了现在的主流电子显示设备，其中包括电视、电脑显示器、手机屏幕及各种嵌入式设备的显示器。图 28-1 是液晶电视与 CRT 电视的外观对比，很明显液晶电视更薄，“时尚”是液晶电视给人的第一印象，而 CRT 电视则感觉很“笨重”。



图 28-1 液晶电视及 CRT 电视

液晶是一种介于固体和液体之间的特殊物质，它是一种有机化合物，常态下呈液态，但是它的分子排列却和固体晶体一样非常规则，因此取名液晶。如果给液晶施加电场，会改变它的分子排列，从而改变光线的传播方向，配合偏振光片，它就具有控制光线透过率的作用，再配合彩色滤光片，改变加给液晶电压大小，就能改变某一颜色透光量的多少，图 28-2 中的就是绿色显示结构。利用这种原理，做出可控红、绿、蓝光输出强度的显示结构，把三种显示结构组成一个显示单位，通过控制红绿蓝的强度，可以使该单位混合输出不同的色彩，这样的一个显示单位被称为像素。

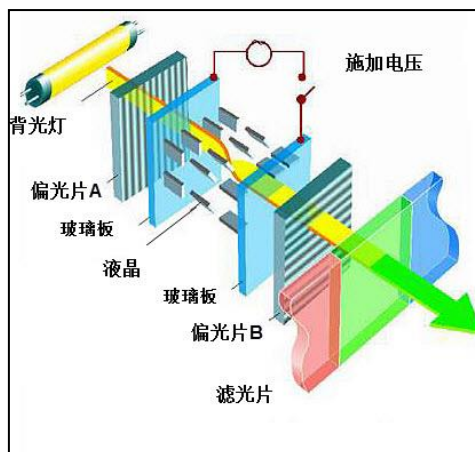


图 28-2 液晶屏的绿色显示结构

注意液晶本身是不发光的，所以需要有一个背光灯提供光源，光线经过一系列处理过程才到输出，所以输出的光线强度是要比光源的强度低很多的，比较浪费能源(当然，比CRT显示器还是节能多了)。而且这些处理过程会导致显示方向比较窄，也就是它的视角较小，从侧面看屏幕会看不清它的显示内容。另外，输出的色彩变换时，液晶分子转动也需要消耗一定的时间，导致屏幕的响应速度低。

28.1.2 LED 和 OLED 显示器

LED点阵显示器不存在以上液晶显示器的问题，LED点阵彩色显示器的单个像素点内包含红绿蓝三色LED灯，显示原理类似我们实验板上的LED彩灯，通过控制红绿蓝颜色的强度进行混色，实现全彩颜色输出，多个像素点构成一个屏幕。由于每个像素点都是LED灯自发光的，所以在户外白天也显示得非常清晰，但由于LED灯体积较大，导致屏幕的像素密度低，所以它一般只适合用于广场上的巨型显示器。相对来说，单色的LED点阵显示器应用得更广泛，如公交车上的信息展示牌、店招等，见图28-3。



图 28-3 LED 点阵彩屏有 LED 单色显示屏

新一代的OLED显示器与LED点阵彩色显示器的原理类似，但由于它采用的像素单元是“有机发光二极管”(Organic Light Emitting Diode)，所以像素密度比普通LED点阵显示器高得多，见图28-5。

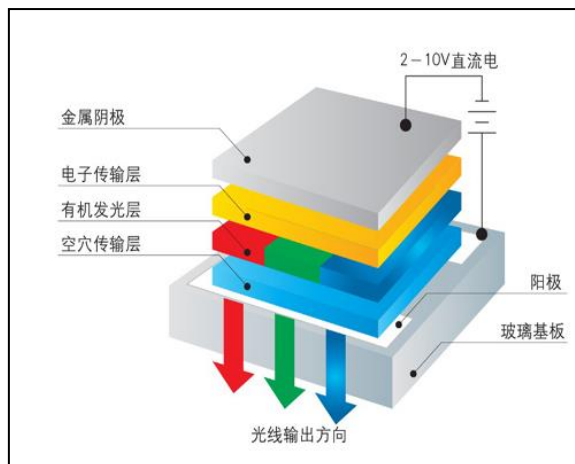


图 28-4 OLED 像素结构

OLED 显示器不需要背光源、对比度高、轻薄、视角广及响应速度快等优点。待到生产工艺更加成熟时，必将取代现在液晶显示器的地位，见图 28-5。



图 28-5 采用 OLED 屏幕的电视及智能手表

28.1.3 显示器的基本参数

不管是哪一种显示器，都有一定的参数用于描述它们的特性，各个参数介绍如下：

(1) 像素

像素是组成图像的最基本单元要素，显示器的像素指它成像最小的点，即前面讲解液晶原理中提到的一个显示单元。

(1) 分辨率

一些嵌入式设备的显示器常常以“行像素值 x 列像素值”表示屏幕的分辨率。如分辨率 800x480 表示该显示器的每一行有 800 个像素点，每一列有 480 个像素点，也可理解为有 800 列，480 行。

(2) 色彩深度

色彩深度指显示器的每个像素点能表示多少种颜色，一般用“位”(bit)来表示。如单色屏的每个像素点能表示亮或灭两种状态(即实际上能显示 2 种颜色)，用 1 个数据位

就可以表示像素点的所有状态，所以它的色彩深度为 1bit，其它常见的显示屏色深为 16bit、24bit。

(3) 显示器尺寸

显示器的大小一般以英寸表示，如 5 英寸、21 英寸、24 英寸等，这个长度是指屏幕对角线的长度，通过显示器的对角线长度及长宽比可确定显示器的实际长宽尺寸。

(4) 点距

点距指两个相邻像素点之间的距离，它会影响画质的细腻度及观看距离，相同尺寸的屏幕，若分辨率越高，则点距越小，画质越细腻。如现在有些手机的屏幕分辨率比电脑显示器的还大，这是手机屏幕点距小的原因；LED 点阵显示屏的点距一般都比较大，所以适合远距离观看。

28.2 液晶控制原理

图 28-6 是两种适合于 STM32 芯片使用的显示屏，我们以它为例讲解控制液晶屏的基本原理。



图 28-6 适合 STM32 控制的显示屏实物图

这个完整的显示屏由液晶显示面板、电容触摸面板以及 PCB 底板构成。图中的触摸面板带有触摸控制芯片，该芯片处理触摸信号并通过引出的信号线与外部器件通讯，触摸面板中间是透明的，它贴在液晶面板上面，一起构成屏幕的主体，触摸面板与液晶面板引出的排线连接到 PCB 底板上，根据实际需要，PCB 底板上可能会带有“液晶控制器芯片”，图中右侧的液晶屏 PCB 上带有 RA8875 液晶控制器。因为控制液晶面板需要比较多的资源，所以大部分低级微控制器都不能直接控制液晶面板，需要额外配套一个专用液晶控制器来处理显示过程，外部微控制器只要把它希望显示的数据直接交给液晶控制器即可。而不带液晶控制器的 PCB 底板，只有小部分的电源管理电路，液晶面板的信号线与外部微控制器相连，直接控制。STM32F429 系列的芯片不需要额外的液晶控制器，也就是说它把专用液晶控制器的功能集成到 STM32F429 芯片内部了，可以理解为电脑的 CPU 集成显卡，它节约了额外的控制器成本。而 STM32F407 系列的芯片由于没有集成液晶控制器到芯片内部，所以它只能驱动自带控制器的屏幕，可以理解为电脑的外置显卡。

总的来说，这两类屏幕的控制框图如图 28-7 所示。

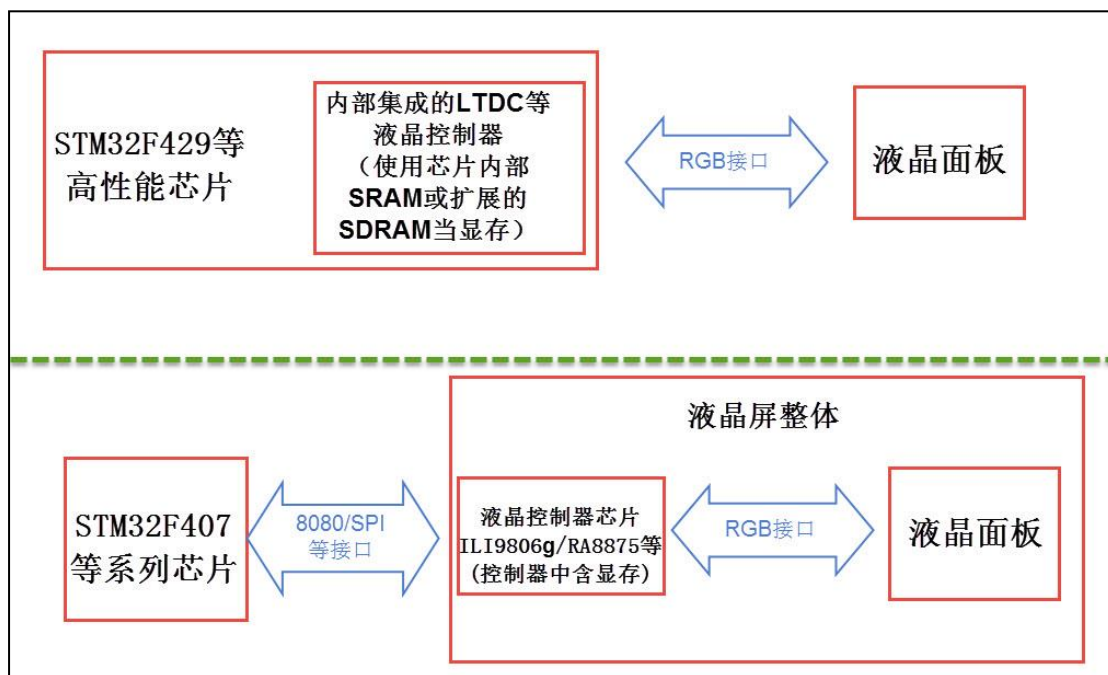


图 28-7 两类液晶屏控制框图

28.2.1 液晶面板的控制信号

本章我们主要讲解如何控制液晶面板，液晶面板的控制信号线即图 28-6 中液晶面板引出的 FPC 排线，其说明见表 28-1，液晶面板通过这些信号线与液晶控制器通讯，使用这种通讯信号的被称作 RGB 接口(RGB Interface)。

表 28-1 液晶面板的信号线

信号名称	说明
R[7:0]	红色数据
G[7:0]	绿色数据
B[7:0]	蓝色数据

CLK	像素同步时钟信号
HSYNC	水平同步信号
VSYNC	垂直同步信号
DE	数据使能信号

(1) RGB 信号线

RGB 信号线各有 8 根，分别用于表示液晶屏一个像素点的红、绿、蓝颜色分量。使用红绿蓝颜色分量来表示颜色是一种通用的做法，打开 Windows 系统自带的画板调色工具，可看到颜色的红绿蓝分量值，见图 28-8。常见的颜色表示会在“RGB”后面附带各个颜色分量值的数据位数，如 RGB565 表示红绿蓝的数据线数分别为 5、6、5 根，一共为 16 个数据位，可表示 2^{16} 种颜色；而这个液晶屏的种颜色分量的数据线都有 8 根，所以它支持 RGB888 格式，一共 24 位数据线，可表示的颜色为 2^{24} 种。

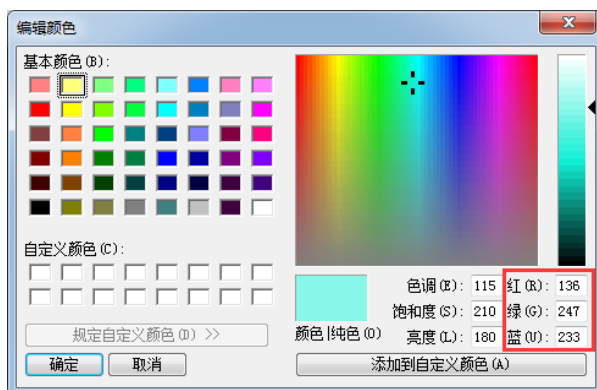


图 28-8 颜色表示法

(2) 同步时钟信号 CLK

液晶屏与外部使用同步通讯方式，以 CLK 信号作为同步时钟，在同步时钟的驱动下，每个时钟传输一个像素点数据。

(3) 水平同步信号 HSYNC

水平同步信号 HSYNC(Horizontal Sync)用于表示液晶屏一行像素数据的传输结束，每传输完成液晶屏的一行像素数据时，HSYNC 会发生电平跳变，如分辨率为 800x480 的显示屏(800 列，480 行)，传输一帧的图像 HSYNC 的电平会跳变 480 次。

(4) 垂直同步信号 VSYNC

垂直同步信号 VSYNC(Vertical Sync)用于表示液晶屏一帧像素数据的传输结束，每传输完成一帧像素数据时，VSYNC 会发生电平跳变。其中“帧”是图像的单位，一幅图像称为一帧，在液晶屏中，一帧指一个完整屏液晶像素点。人们常常用“帧/秒”来表示液晶屏的刷新特性，即液晶屏每秒可以显示多少帧图像，如液晶屏以 60 帧/秒的速率运行时，VSYNC 每秒钟电平会跳变 60 次。

(5) 数据使能信号 DE

数据使能信号 DE(Data Enable)用于表示数据的有效性，当 DE 信号线为高电平时，RGB 信号线表示的数据有效。

28.2.2 液晶数据传输时序

通过上述信号线向液晶屏传输像素数据时，各信号线的时序见图 28-9。图中表示的是向液晶屏传输一帧图像数据的时序，中间省略了多行及多个像素点。

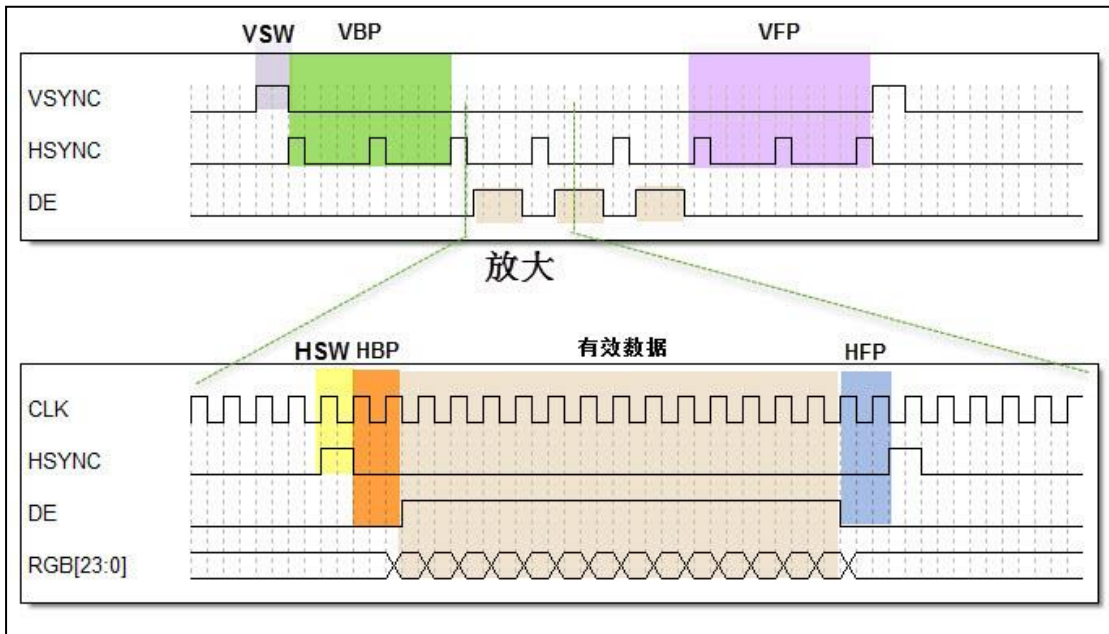


图 28-9 液晶时序图

液晶屏显示的图像可看作一个矩形，结合图 28-10 来理解。液晶屏有一个显示指针，它指向将要显示的像素。显示指针的扫描方向从左到右、从上到下，一个像素点一个像素点地描绘图形。这些像素点的数据通过 RGB 数据线传输至液晶屏，它们在同步时钟 CLK 的驱动下一个一个地传输到液晶屏中，交给显示指针，传输完成一行时，水平同步信号 HSYNC 电平跳变一次，而传输完一帧时 VSYNC 电平跳变一次。

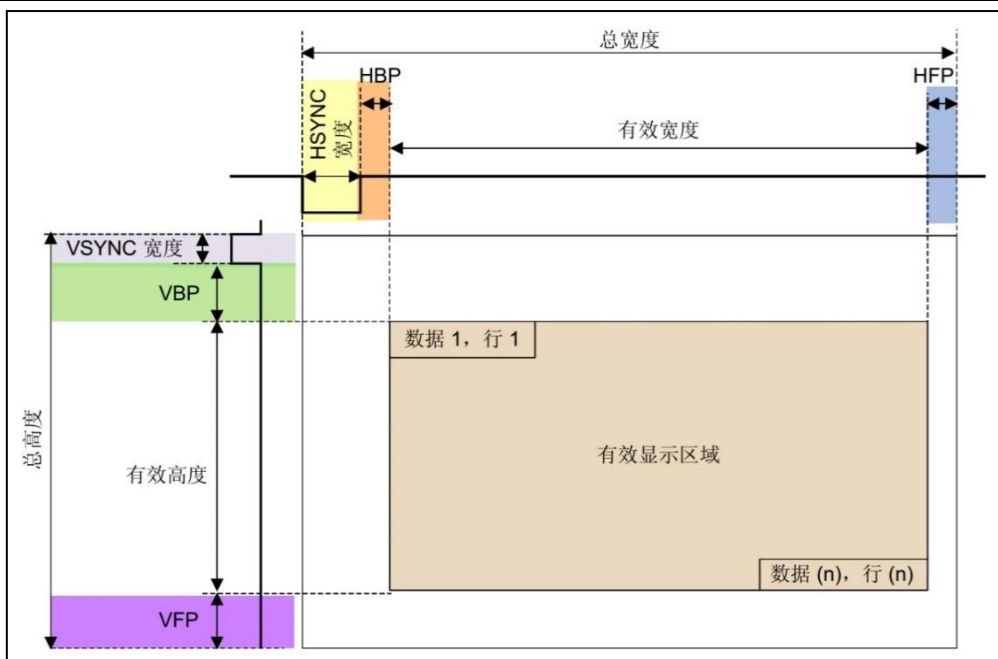


图 28-10 液晶数据传输图解

但是，液晶显示指针在行与行之间，帧与帧之间切换时需要延时，而且 HSYNC 及 VSYNC 信号本身也有宽度，这些时间参数说明见表 28-2。

表 28-2 液晶通讯中的时间参数

时间参数	参数说明
VBP (vertical back porch)	表示在一帧图像开始时，垂直同步信号以后的无效的行数
VFP (vertical front porch)	表示在一帧图像结束后，垂直同步信号以前的无效的行数
HBP (horizontal back porch)	表示从水平同步信号开始到一行的有效数据开始之间的 CLK 的个数
HFP (horizontal front porch)	表示一行的有效数据结束到下一个水平同步信号开始之间的 CLK 的个数
VSW (vertical sync width)	表示垂直同步信号的宽度，单位为行
HSW (horizontal sync width)	表示水平同步信号的宽度，单位为同步时钟 CLK 的个数

在这些时间参数控制的区域，数据使能信号线“DE”都为低电平，RGB 数据线的信号无效，当“DE”为高电平时，表示的数据有效，传输的数据会直接影响液晶屏的显示区域。

28.2.3 显存

液晶屏中的每个像素点都是数据，在实际应用中需要把每个像素点的数据缓存起来，再传输给液晶屏，一般会使用 SRAM 或 SDRAM 性质的存储器，而这些专门用于存储显示数据的存储器，则被称为显存。显存一般至少要能存储液晶屏的一帧显示数据，如分辨率为 800x480 的液晶屏，使用 RGB888 格式显示，它的一帧显示数据大小为：
 $3 \times 800 \times 480 = 1152000$ 字节；若使用 RGB565 格式显示，一帧显示数据大小为：
 $2 \times 800 \times 480 = 768000$ 字节。

一般来说，外置的液晶控制器会自带显存，而像 STM32F429 等集成液晶控制器的芯片可使用内部 SRAM 或外扩 SDRAM 用于显存空间。

28.3 野火 4.5 寸液晶屏简介

28.3.1 4.5 寸电容触摸屏实物

上面讲解的屏幕其液晶控制器与液晶屏是完全分离的，且具有带控制器和不带控制器的版本，易于理解，下面我们再来分析实验板标配的分辨率为 854*480 的 4.5 寸电阻触摸液晶屏，见图 28-11。

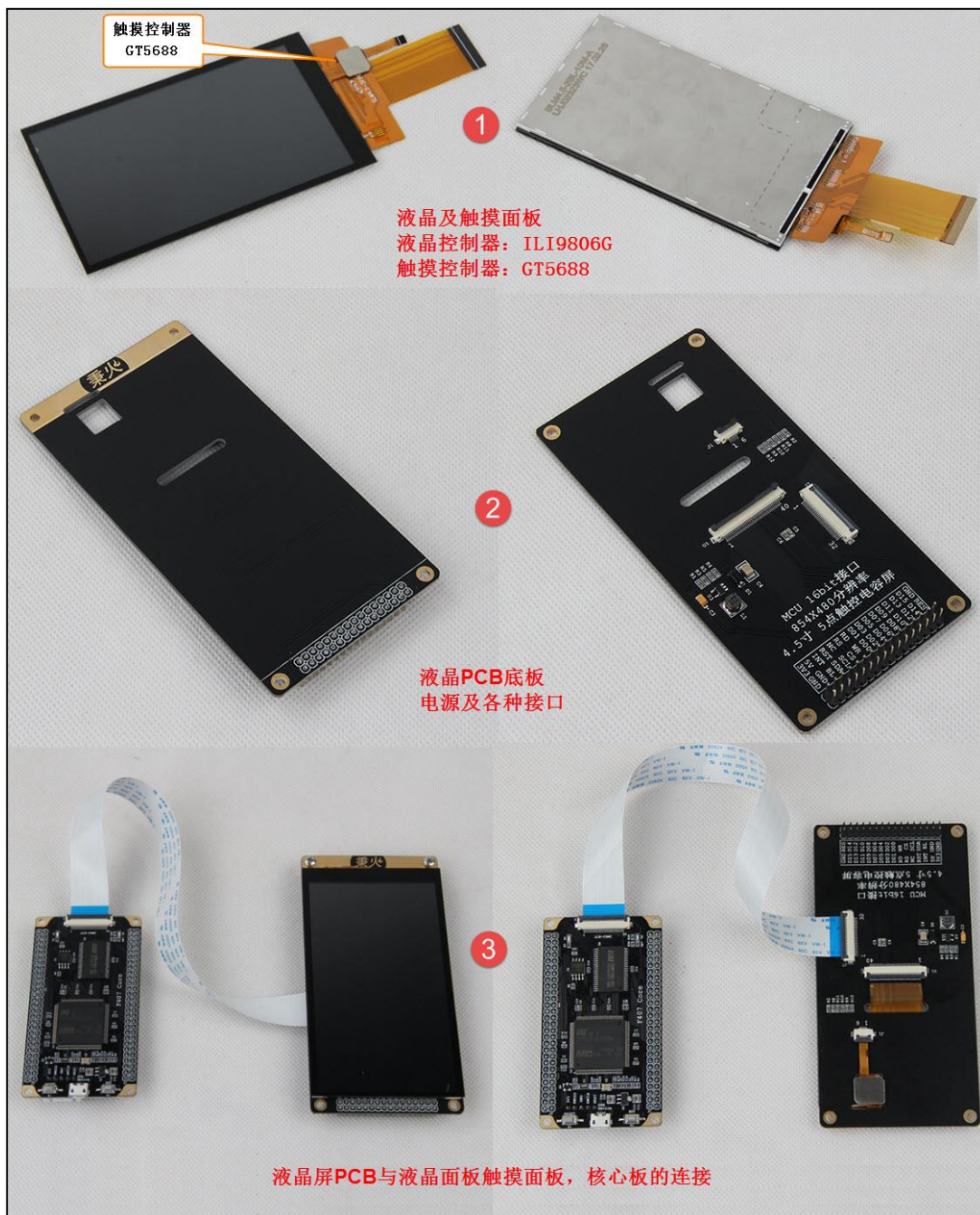


图 28-11 实验板标配的 3.2 寸电阻触摸屏

图中的标号③部分是液晶屏幕的整体，通过引出的排针或 FPC 排线接入到实验板上可对它进行控制，它分为标号①的液晶触摸面板和标号②的 PCB 底板两部分。

标号①处的液晶触摸面板由液晶屏和触摸面板组成，触摸面板和液晶面板是两个独立的部件，生产厂商把这两个部件帖在了一起，形成整体。

触摸面板引出的信号线引入到 GT5688 芯片，该芯片会对触摸信号进行采集并简化信号输出给外部控制器。

触摸面板的下方即为液晶面板，在它的内部包含了一个型号为 ILI9806G 的液晶控制器芯片(由于集成度高，所以图中无法看见)，该液晶控制器使用 8080 接口与单片机通讯，图中液晶面板引出的 FPC 信号线即 8080 接口(RGB 接口已在内部直接与 ILI9806G 相连)，且控制器中包含有显存，单片机把要显示的数据通过引出的 8080 接口发送到液晶控制器，这些数据会被存储到它内部的显存中，然后液晶控制器不断把显存的内容刷新到液晶面板，显示内容。

标号②处的是 PCB 底板，它主要包含了液晶屏的电源控制电路，另外，PCB 底板与液晶触摸面板通过 FPC 排线座连接，然后引出到排针，方便与实验板的排母连接，也可以使用 FPC 接口与核心板连接。

28.3.2 ILI9806G 液晶控制器简介

本液晶屏内部包含有一个液晶控制芯片 ILI9806G，它的内部结构非常复杂，见图 28-12。该芯片最核心部分是位于中间的 GRAM(Graphics RAM)，它就是显存。GRAM 中每个存储单元都对应着液晶面板的一个像素点。它右侧的各种模块共同作用把 GRAM 存储单元的数据转化成液晶面板的控制信号，使像素点呈现特定的颜色，而像素点组合起来则成为一幅完整的图像。

框图的左上角为 ILI9806G 的主要控制信号线和配置引脚，根据其不同状态设置可以使芯片工作在不同的模式，如每个像素点的位数是 8、16 还是 24 位；可配置使用 SPI 接口、8080 接口还是 RGB 接口与 MCU 进行通讯。MCU 通过 SPI、8080 接口或 RGB 接口与 ILI9806G 进行通讯，从而访问它的控制寄存器(CR)、地址计数器(AC)、及 GRAM。

在 GRAM 的上侧还有一个 LED 控制器(LED Controller)。LCD 为非发光性的显示装置，它需要借助背光源才能达到显示功能，LED 控制器就是用来控制液晶屏中的 LED 背光源。

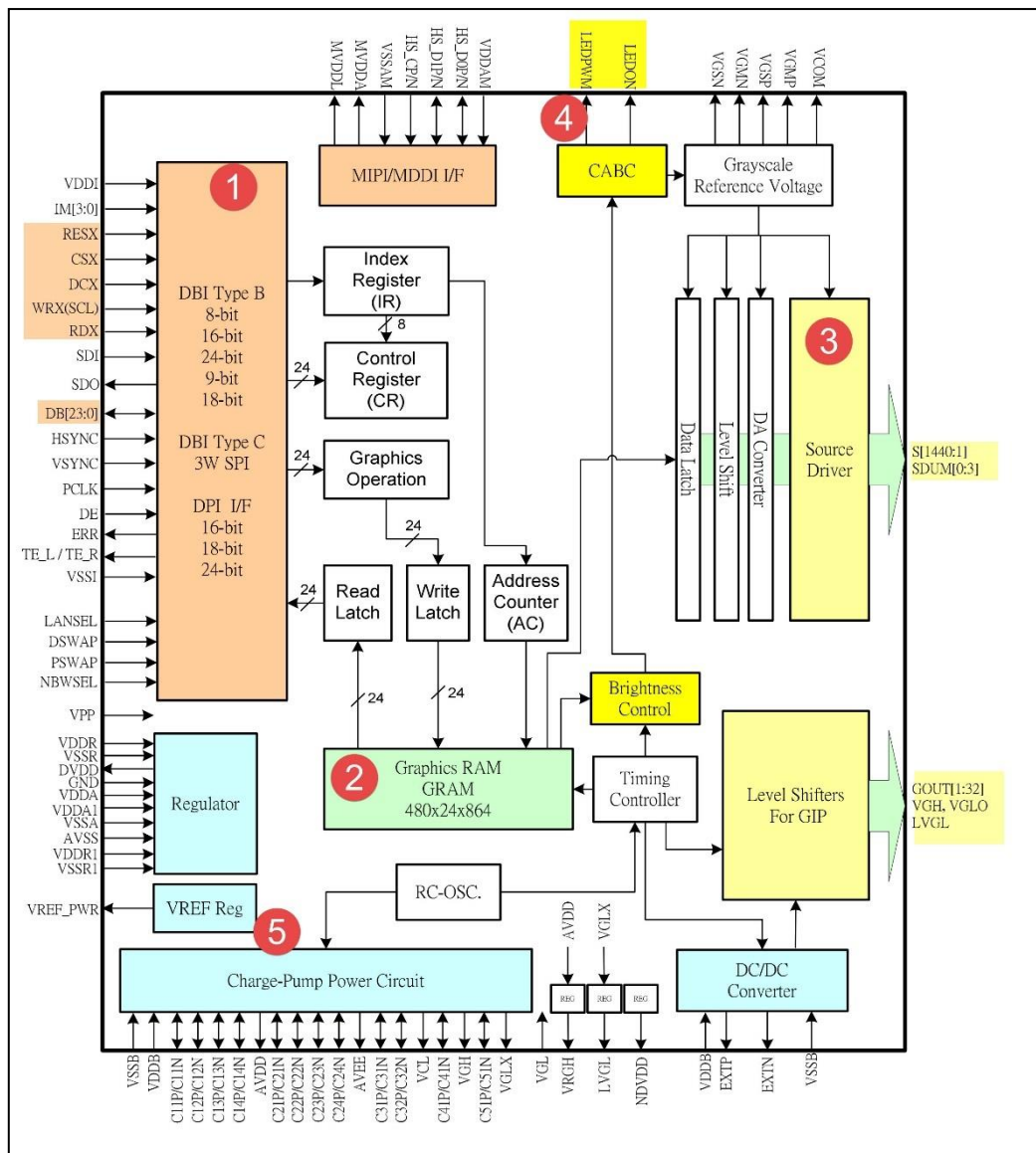


图 28-12 ILI9806G 控制器内部框图

28.3.3 液晶屏的信号线及 8080 时序

ILI9806G 控制器根据自身的 IM[3:0]信号线电平决定它与 MCU 的通讯方式，它本身支持 SPI 及 8080 通讯方式，本示例中液晶屏的 ILI9806G 控制器在出厂前就已经按固定配置好(内部已连接硬件电路)，它被配置为通过 8080 接口通讯，使用 16 根数据线的 RGB565 格式，且背光 LED 引脚不与 ILI9806G 相连，而是直接引出到排针供外部控制器控制。内部硬件电路连接完，剩下的其它信号线被引出到 FPC 排线，最后该排线由 PCB 底板引出到排针，排针再与实验板上的 STM32 芯片连接，引出的排针信号线见图 28-13。

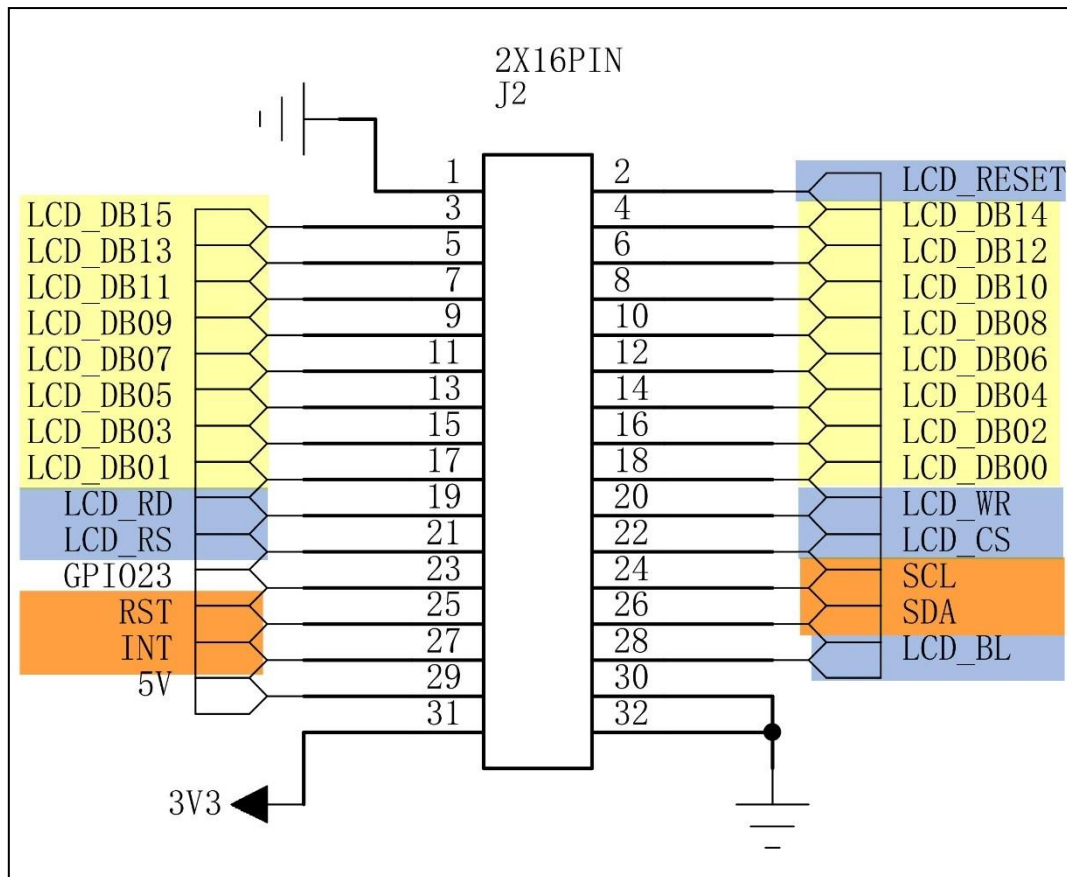


图 28-13 液晶屏引出的信号线

这些信号线的说明见表 28-3。

表 28-3 液晶屏引出的信号线说明

信号线	ILI9806G 对应的信号线	说明
LCD_DB[15:0]	D[15:0]	数据信号
LCD_RD	RDX	读数据信号，低电平有效
LCD_RS	D/CX	数据/命令信号，高电平时，D[15:0]表示的是数据(RGB 像素数据或命令数据)，低电平时 D[15:0]表示控制命令
LCD_RESET	RESX	复位信号，低电平有效
LCD_WR	WRX	写数据信号，低电平有效
LCD_CS	CSX	片选信号，低电平有效
LCD_BK	-	背光信号，低电平点亮
GPIO23	-	空引脚，不需要连接
RST	与触摸 IC 相连	触摸 IC 的复位引脚
INT	与触摸 IC 相连	触摸 IC 的中断信号引脚
SCL	与触摸 IC 相连	触摸 IC 的 IIC 总线的时钟信号
SDA	与触摸 IC 相连	触摸 IC 的 IIC 总线的数据信号

这些信号线即 8080 通讯接口，带 X 的表示低电平有效，STM32 通过该接口与 ILI9806G 芯片进行通讯，实现对液晶屏的控制。通讯的内容主要包括命令和显存数据，显存数据即各个像素点的 RGB565 内容；命令是指对 ILI9806G 的控制指令，MCU 可通过 8080 接口发送命令编码控制 ILI9806G 的工作方式，例如复位指令、设置光标指令、睡眠

模式指令等等，具体的指令在《ILI9806G-Data Sheet.pdf》数据手册均有详细说明。写命令时序图见图 28-14。

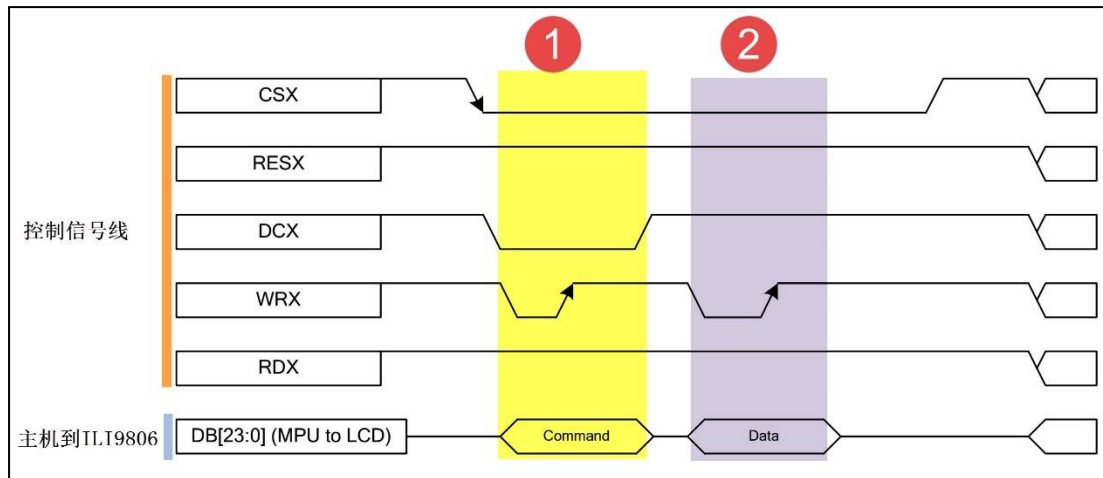


图 28-14 使用 8080 接口写命令时序

由图可知，写命令时序由片选信号 CSX 拉低开始，对数据/命令选择信号线 D/CX 也置低电平表示写入的是命令地址(可理解为命令编码，如软件复位命令：0x01)，以写信号 WRX 为低，读信号 RDX 为高表示数据传输方向为写入，同时，在数据线 D[23:0](或 D[15:0])输出命令地址，在第二个传输阶段传送的是命令的参数，所以 D/CX 要置高电平，表示写入的是命令数据，命令数据是某些指令带有的参数，如复位指令编码为 0x01，它后面可以带一个参数，该参数表示多少秒后复位(实际的复位命令不含参数，此处只是为了讲解指令编码与参数的区别)。

当需要把像素数据写入 GRAM 时，过程很类似，把片选信号 CSX 拉低后，再把数据/命令选择信号线 D/CX 置为高电平，这时由 D[23:0]传输的数据则会被 ILI9806G 保存至它的 GRAM 中。

28.4 使用 STM32 的 FSMC 模拟 8080 接口时序

ILI9806G 的 8080 通讯接口时序可以由 STM32 使用普通 I/O 接口进行模拟，但这样效率太低，STM32 提供了一种特别的控制方法——使用 FSMC 接口实现 8080 时序。

28.4.1 FSMC 简介

在前面的《FSMC—扩展外部 SRAM》章节中了解到 STM32 的 FSMC 外设可以用于控制扩展的外部存储器，而 MCU 对液晶屏的操作实际上就是把显示数据写入到显存中，与控制存储器非常类似，且 8080 接口的通讯时序完全可以使用 FSMC 外设产生，因而非常适合使用 FSMC 控制液晶屏。

FSMC 外设的结构见图 28-15。

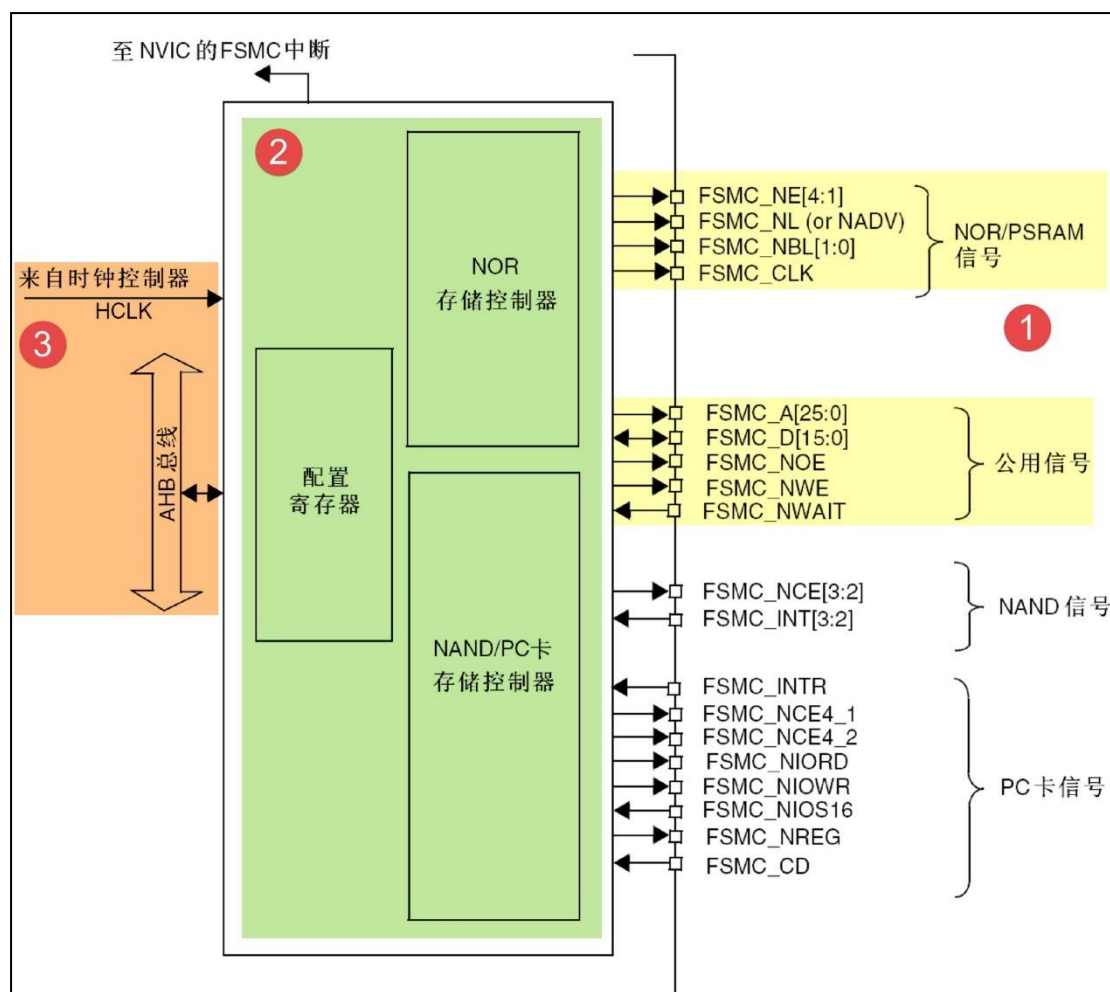


图 28-15 FSMC 结构图

控制 LCD 时，是使用 FSMC 的 NOR/PSRAM 模式的，且与前面使用 FSMC 控制 SRAM 的稍有不同，控制 SRAM 时使用的是模式 A，而控制 LCD 使用的是与 NOR FLASH 一样的模式 B，所以我们重点分析框图中 NOR FLASH 控制信号线部分。控制 NOR FLASH 主要使用到表 28-4 的信号线：

表 28-4 FSMC 控制 NOR FLASH 的信号线

FSMC 信号名	信号方向	功能
CLK	输出	时钟(同步突发模式使用)
A[25:0]	输出	地址总线
D[15:0]	输入/输出	双向数据总线
NE[x]	输出	片选, x = 1...4
NOE	输出	输出使能
NWE	输出	写使能
NWAIT	输入	NOR 闪存要求 FSMC 等待的信号
NADV	输出	地址、数据线复用时的锁存信号

在控制 LCD 时，使用的是类似异步、地址与数据线独立的 NOR FLASH 控制方式，所以实际上 CLK、NWAIT、NADV 引脚并没有使用到。

FSMC NOR/PSRAM 中的模式 B 的写时序见图 28-16。

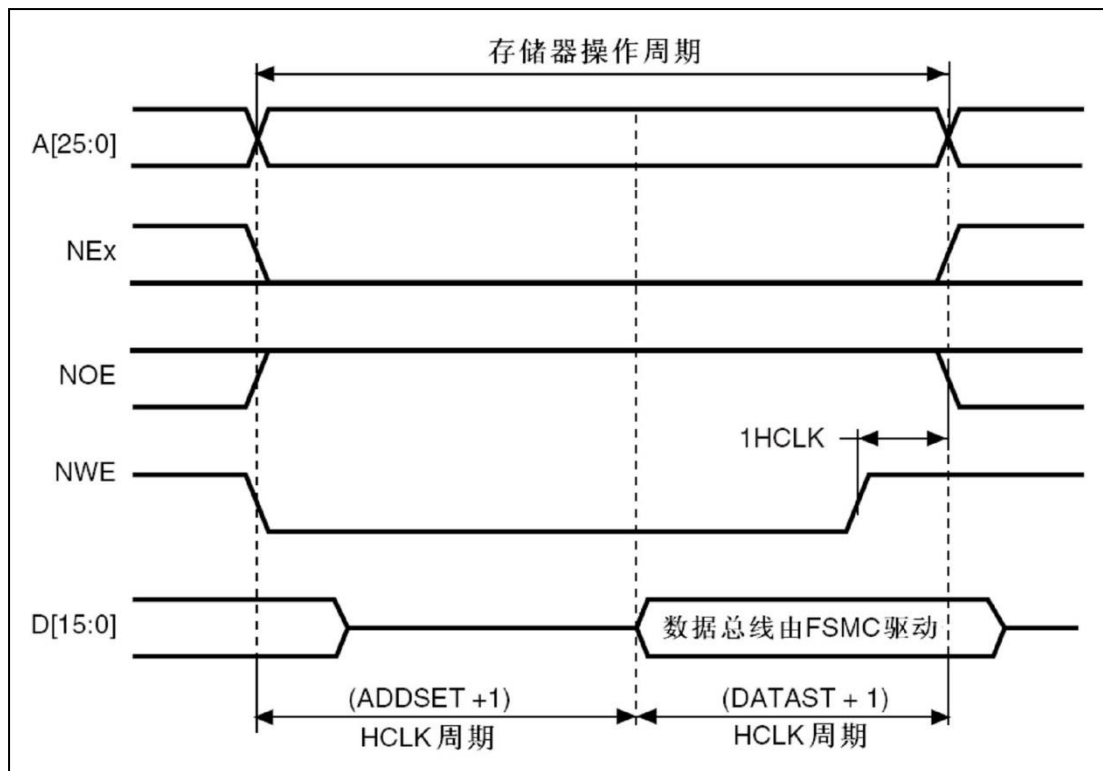


图 28-16 FSMC 写 NOR 时序图（模式 B）

根据 STM32 对寻址空间的地址映射，地址 0x6000 0000 ~ 0x9FFF FFFF 是映射到外部存储器的，而其中的 0x6000 0000 ~ 0x6FFF FFFF 则是分配给 NOR FLASH、PSRAM 这类可直接寻址的器件。

当 FSMC 外设被配置成正常工作，并且外部接了 NOR FLASH 时，若向 0x60000000 地址写入数据如 0xABCD，FSMC 会自动在各信号线上产生相应的电平信号，写入数据。FSMC 会控制片选信号 NE1 选择相应的 NOR 芯片，然后使用地址线 A[25:0] 输出 0x60000000，在 NWE 写使能信号线上发出低电平的写使能信号，而要写入的数据信号 0xABCD 则从数据线 D[15:0] 输出，然后数据就被保存到 NOR FLASH 中了。

28.4.2 用 FSMC 模拟 8080 时序

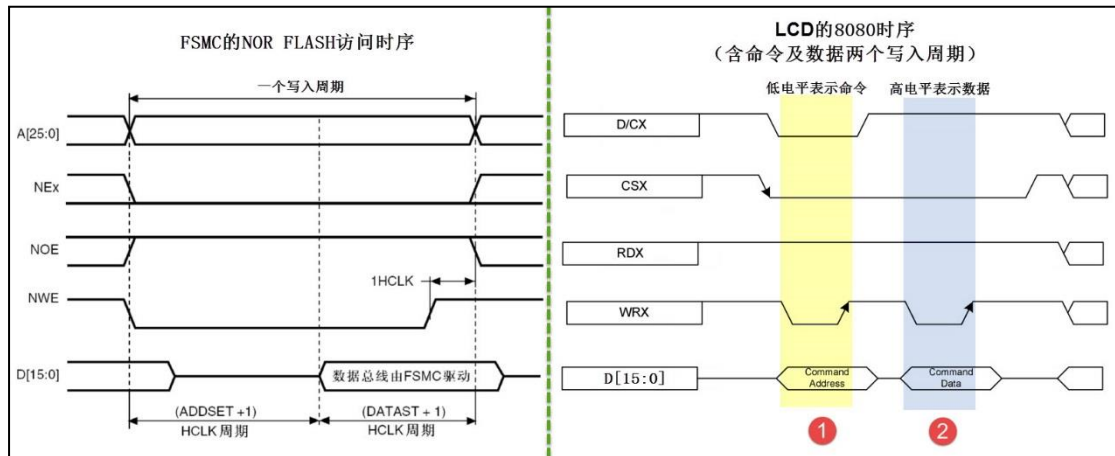


图 28-17FSMC 模式 B 时序与 8080 时序对比（写过程）

见表 28-5，对比 FSMC NOR/PSRAM 中的模式 B 时序与 ILI9806G 液晶控制器芯片使用的 8080 时序可发现，这两个时序是十分相似的(除了 FSMC 的地址线 A 和 8080 的 D/CX 线，可以说是完全一样)，它们的信号线对比见表 28-5。

表 28-5 FSMC 的 NOR 与 8080 信号线对比

FSMC-NOR 信号线	功能	8080 信号线	功能
NEx	片选信号	CSX	片选信号
NWR	写使能	WRX	写使能
NOE	读使能	RDX	读使能
D[15:0]	数据信号	D[15:0]	数据信号
A[25:0]	地址信号	D/CX	数据/命令选择

对于 FSMC 和 8080 接口，前四种信号线都是完全一样的，仅仅是 FSMC 的地址信号线 A[25:0]与 8080 的数据/命令选择线 D/CX 有区别。而对于 D/CX 线，它为高电平的时候表示数值，为低电平的时候表示命令，如果能使用 FSMC 的 A 地址线根据不同的情况产生对应的电平，那么就完全可以使用 FSMC 来产生 8080 接口需要的时序了。

为了模拟出 8080 时序，我们可以把 FSMC 的 A0 地址线(也可以使用其它 A1/A2 等地址线)与 ILI9806G 芯片 8080 接口的 D/CX 信号线连接，那么当 A0 为高电平时(即 D/CX 为高电平)，数据线 D[15:0]的信号会被 ILI9806G 理解为数值，若 A0 为低电平时(即 D/CX 为低电平)，传输的信号则会被理解为命令。

由于 FSMC 会自动产生地址信号，当使用 FSMC 向 0x6xxx xxx1、0x6xxx xxx3、0x6xxx xxx5…这些奇数地址写入数据时，地址最低位的值均为 1，所以它会控制地址线 A0(D/CX)输出高电平，那么这时通过数据线传输的信号会被理解为数值；若向 0x6xxx xxx0、0x6xxx xxx2、0x6xxx xxx4…这些偶数地址写入数据时，地址最低位的值均为 0，所以它会控制地址线 A0(D/CX)输出低电平，因此这时通过数据线传输的信号会被理解为命令，见表 28-6。

表 28-6 使用 FSMC 输出地址示例

地址	地址的二进制值	A0(D/CX)的电平	控制 ILI9806G 时
----	---------	-------------	---------------

	(仅列出低四位)		的意义
0x6xxx xxx1	0001	1 高电平	D 数值
0x6xxx xxx3	0011	1 高电平	D 数值
0x6xxx xxx5	0101	1 高电平	D 数值
0x6xxx xxx0	0000	0 低电平	C 命令
0x6xxx xxx2	0010	0 低电平	C 命令
0x6xxx xxx4	0100	0 低电平	C 命令

有了这个基础，只要配置好 FSMC 外设，然后在代码中利用指针变量，向不同的地址单元写入数据，就能够由 FSMC 模拟出的 8080 接口向 ILI9806G 写入控制命令或 GRAM 的数据了。

注意：在实际控制时，以上地址计算方式还不完整，还需要注意 HADDR 内部地址与 FSMC 地址信号线的转换，关于这部分内容在代码讲解时再详细举例说明。

28.5 NOR FLASH 时序结构体

在讲解程序前，再来了解一下与 FSMC NOR FLASH 控制相关的结构体。

与控制 SRAM 时一样，控制 FSMC 使用 NOR FLASH 存储器时主要是配置时序寄存器以及控制寄存器，利用 ST 标准库的时序结构体以及初始化结构体可以很方便地写入参数。

NOR FLASH 时序结构体的成员见代码清单 25-1。

代码清单 28-1 NOR FLASH 时序结构体 FSMC_NORSRAMTimingInitTypeDef

```

1 typedef struct
2 {
3     uint32_t FSMC_AddressSetupTime;           /*地址建立时间, 0-0xF 个 HCLK 周期*/
4     uint32_t FSMC_AddressHoldTime;            /*地址保持时间, 0-0xF 个 HCLK 周期*/
5     uint32_t FSMC_DataSetupTime;              /*地址建立时间, 0-0xF 个 HCLK 周期*/
6     uint32_t FSMC_BusTurnAroundDuration; /*总线转换周期, 0-0xF 个 HCLK 周期, 在 NOR FLASH */
7     uint32_t FSMC_CLKDivision; /*时钟分频因子, 1-0xF, 若控制异步存储器, 本参数无效 */
8     uint32_t FSMC_DataLatency;              /*数据延迟时间, 若控制异步存储器, 本参数无效 */
9     uint32_t FSMC_AccessMode;                /*设置访问模式 */
10 }FSMC_NORSRAMTimingInitTypeDef;

```

这个结构体与 SRAM 中的时序结构体完全一样，以下仅列出控制 NOR FLASH 时使用模式 B 用到的结构体成员说明：

(1) FSMC_AddressSetupTime

本成员设置地址建立时间，即 FSMC 读写时序图 28-16 中的 ADDSET 值，它可以被设置为 0-0xF 个 HCLK 周期数，按 STM32 标准库的默认配置，HCLK 的时钟频率为 168MHz，即一个 HCLK 周期为 1/168 微秒。

(2) FSMC_DataSetupTime

本成员设置数据建立时间，即 FSMC 读写时序图 28-16 中的 DATAST 值，它可以被设置为 0-0xF 个 HCLK 周期数。

(3) FSMC_AccessMode

本成员设置存储器访问模式，不同的模式下 FSMC 访问存储器地址时引脚输出的时序不一样，可选 FSMC_AccessMode_A/B/C/D 模式。控制异步 NOR FLASH 时使用 B 模式。

这个 FSMC_NORSRAMTimingInitTypeDef 时序结构体配置的延时参数，将作为下一节的 FSMC NOR FLASH 初始化结构体的一个成员。

28.6 FSMC 初始化结构体

FSMC 控制 NOR FLASH 相关的结构体，初始化结构体见代码清单 28-2。

代码清单 28-2 NOR FLASH 初始化结构体 FSMC_NORSRAMInitTypeDef

```

1  /**
2   * @brief  FSMC NOR/SRAM Init structure definition
3   */
4  typedef struct
5  {
6      uint32_t FSMC_Bank;          /*设置要控制的 Bank 区域 */
7      uint32_t FSMC_DataAddressMux; /*设置地址总线与数据总线是否复用 */
8      uint32_t FSMC_MemoryType;    /*设置存储器的类型 */
9      uint32_t FSMC_MemoryDataWidth; /*设置存储器的数据宽度*/
10     uint32_t FSMC_BurstAccessMode; /*设置是否支持突发访问模式，只支持同步类型的存储器 */
11     uint32_t FSMC_AsynchronousWait; /*设置是否使能在同步传输时的等待信号，*/
12     uint32_t FSMC_WaitSignalPolarity; /*设置等待信号的极性*/
13     uint32_t FSMC_WrapMode;        /*设置是否支持对齐的突发模式 */
14     uint32_t FSMC_WaitSignalActive; /*配置等待信号在等待前有效还是等待期间有效 */
15     uint32_t FSMC_WriteOperation;  /*设置是否写使能 */
16     uint32_t FSMC_WaitSignal;      /*设置是否使能等待状态插入 */
17     uint32_t FSMC_ExtendedMode;    /*设置是否使能扩展模式 */
18     uint32_t FSMC_WriteBurst;      /*设置是否使能写突发操作*/
19     /*当不使用扩展模式时，本参数用于配置读写时序，否则用于配置读时序*/
20     FSMC_NORSRAMTimingInitTypeDef* FSMC_ReadWriteTimingStruct;
21     /*当使用扩展模式时，本参数用于配置写时序*/
22     FSMC_NORSRAMTimingInitTypeDef* FSMC_WriteTimingStruct;
23 }FSMC_NORSRAMInitTypeDef;

```

这部分内容与 FSMC 控制 SRAM 时完全一致，此处仅列出与 NOR FLASH 模式 B 相关的一些结构体成员的说明：

(1) FSMC_Bank

本成员用于选择 FSMC 映射的存储区域，它的可选参数以及相应的内核地址映射范围见表 27-4。

表 28-7 可以选择的存储器区域及区域对应的地址范围

可以输入的宏	对应的地址区域
FSMC_Bank1_NORSRAM1	0x60000000-0x63FFFFFF
FSMC_Bank1_NORSRAM2	0x64000000-0x67FFFFFF
FSMC_Bank1_NORSRAM3	0x68000000-0x6BFFFFFF
FSMC_Bank1_NORSRAM4	0x6C000000-0x6FFFFFFF

(2) FSMC_MemoryType

本成员用于设置要控制的存储器类型，它支持控制的存储器类型为 SRAM、PSRAM 以及 NOR FLASH(FSMC_MemoryType_SRAM/PSRAM/NOR)。

(3) FSMC_MemoryDataWidth

本成员用于设置要控制的存储器的数据宽度，可选择设置成 8 或 16 位 (FSMC_MemoryDataWidth_8b /16b)。

(4) FSMC_WriteOperation

这个成员用于设置是否写使能(FSMC_WriteOperation_Enable /Disable)，禁止写使能的话 FSMC 只能从存储器中读取数据，不能写入。

(5) FSMC_ExtendedMode

本成员用于设置是否使用扩展模式(FSMC_ExtendedMode_Enable/Disable)，在非扩展模式下，对存储器读写的时序都只使用 FSMC_BCR 寄存器中的配置，即下面的 FSMC_ReadWriteTimingStruct 结构体成员；在扩展模式下，对存储器的读写时序可以分开配置，读时序使用 FSMC_BCR 寄存器，写时序使用 FSMC_BWTR 寄存器的配置，即后面的 FSMC_WriteTimingStruct 结构体成员。

(6) FSMC_ReadWriteTimingStruct

本成员是一个指针，赋值时使用上一小节中讲解的时序结构体

FSMC_NORSRAMInitTypeDef 设置，当不使用扩展模式时，读写时序都使用本成员的参数配置。

(7) FSMC_WriteTimingStruct

同样地，本成员也是一个时序结构体的指针，只有当使用扩展模式时，本配置才有效，它是写操作使用的时序。

对本结构体赋值完成后，调用 FSMC_NORSRAMInit 库函数即可把配置参数写入到 FSMC_BCR 及 FSMC_BTR/BWTR 寄存器中。

28.7 FSMC—液晶显示实验

本小节讲解如何使用 FSMC 外设控制实验板配套的 4.5 寸 ILI9806G 液晶屏，见图 28-18，该液晶屏的分辨率为 854x480，支持 RGB565 格式。

学习本小节内容时，请打开配套的“FSMC—液晶显示英文”工程配合阅读。

28.7.1 硬件设计



图 28-18 液晶屏实物图

图 28-18 液晶屏背面的 PCB 电路对应图 28-19、图 28-20、图 28-21 中的原理图，分别是屏幕 PCB 底板原理图、触摸部分原理图、液晶排针接口线序图。

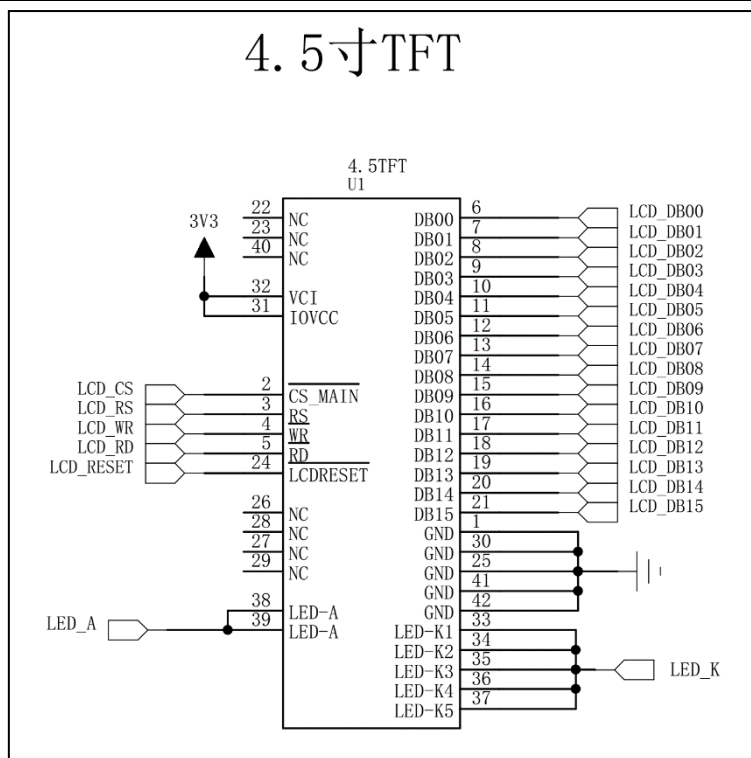


图 28-19 屏幕 PCB 底板原理图(截图于《4.5 寸液晶原理图.pdf》)

屏幕的 PCB 底板引出的信号线会通过 PCB 底板上的 FPC 接口与液晶面板连接，这些信号包括液晶控制相关的 CS、RS 等信号及 DB0-DB15 数据线，其中 RS 引脚以高电平表示传输数据，低电平表示传输命令；另外还有引出 LCD_K 引脚用于控制屏幕的背光供电，可以通过该引脚控制背光的强度，该引脚为低电平时打开背光。除液晶部分的 FPC 接口外，PCB 底板还有一个连接到电容触摸屏控制器的 FPC 接口，用于检测触摸信号，其原理图见图 28-20。

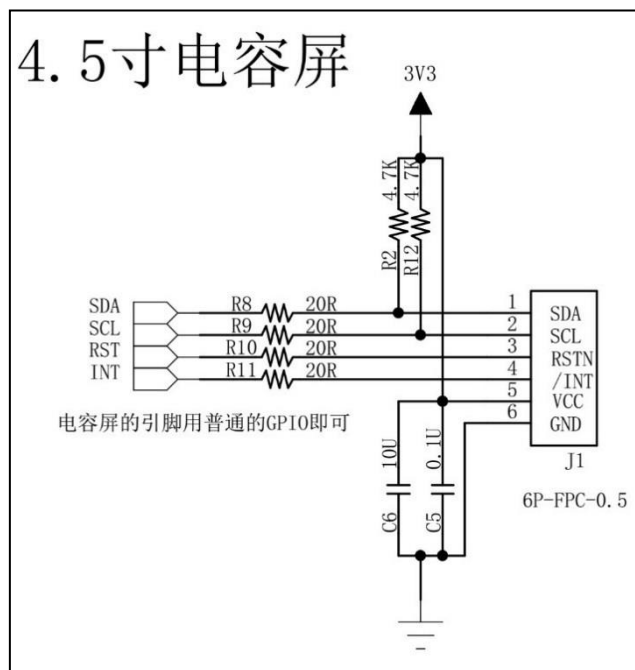


图 28-20 屏幕 PCB 底板的触摸部分原理图(截图于《4.5 寸液晶原理图.pdf》)

触摸检测的主体是型号为 GT5688 的芯片，它接收触摸屏的信号进行处理，把触摸信息使用 I2C 接口输出到 STM32 等控制器，在触摸屏章节将会详细讲解其检测原理。

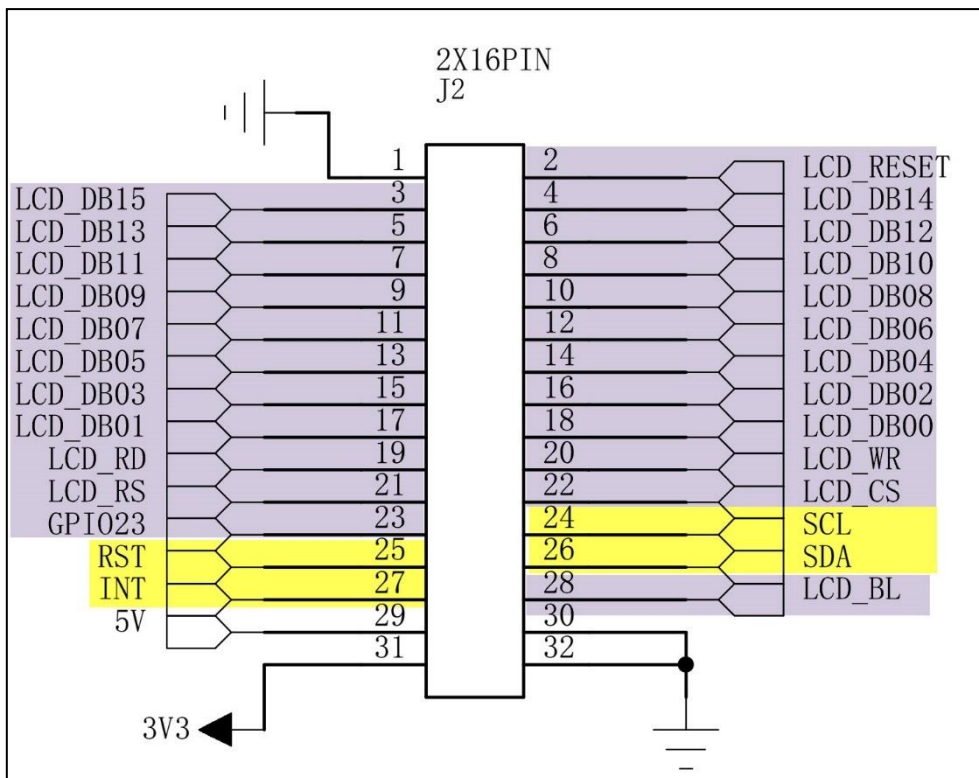


图 28-21 液晶屏接口(截图于《4.5 寸液晶原理图.pdf》)

图 28-21 表示的是 PCB 底板引出的排针线序，屏幕整体通过这些引出的排针与开发板或其它控制器连接。

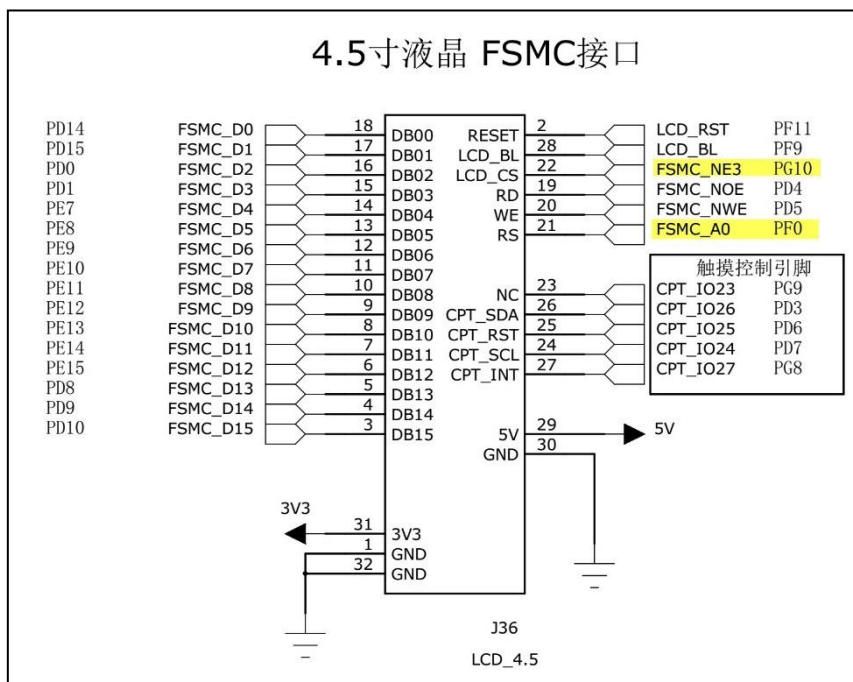


图 28-22 开发板与屏幕的连接的信号说明(截图于《F407 开发板底板原理图》)

图 28-22 是野火 F407 开发板上的液晶排母接口原理图，它说明了配套的 4.5 寸屏幕接入到开发板上时的信号连接关系。其中请着重关注图中液晶屏 LCD_CS 及 LCD_RS(即 DC 引脚)与 FSMC 存储区选择引脚 FSMC_NE 及地址信号 FSMC_A 的编号，它们会决定 STM32 要使用什么内存地址来控制与液晶屏的通讯。

以上原理图可查阅《4.5 寸液晶原理图.pdf》及《F407 开发板底板原理图》文档获知，若您使用的液晶屏或实验板不一样，请根据实际连接的引脚修改程序。

28.7.2 软件设计

为了使工程更加有条理，我们把 LCD 控制相关的代码独立分开存储，方便以后移植。在“USART—串口通讯”工程的基础上新建“bsp_ili9806g_lcd.c”及“bsp_ili9806g_lcd.h”文件，这些文件也可根据您的喜好命名，它们不属于 STM32 标准库的内容，是由我们自己根据应用需要编写的。

1. 编程要点

- (1) 初始化通讯使用的目标引脚及端口时钟；
- (2) 使能 FSMC 外设的时钟；
- (3) 配置 FSMC 为异步 NOR FLASH 模式以仿真 8080 时序；
- (4) 建立机制使用 FSMC 向液晶屏发送命令及数据；
- (5) 发送控制命令初始化液晶屏；
- (6) 编写液晶屏的绘制像素点函数；
- (7) 利用描点函数制作各种不同的液晶显示应用。

2. 代码分析

液晶 LCD 硬件相关宏定义

我们把 FSMC 控制液晶屏硬件相关的配置都以宏的形式定义到“bsp_ili9806g_lcd.h”文件中，见代码清单 28-3。

代码清单 28-3 FSMC 硬件配置相关的宏(省略了部分数据线)

```
1 //由片选引脚决定的 NOR/SRAM 块
2 #define      FSMC_Bank1_NORSRAMx          FSMC_Bank1_NORSRAM3
3
4 /***** ILI9806G 显示屏 8080 通讯引脚定义 *****/
5 /*****控制信号线*****/
6 #define      FSMC_AF                      GPIO_AF_FSMC
7 //片选, 选择 NOR/SRAM 块
8 #define      ILI9806G_CS_CLK              RCC_AHB1Periph_GPIOG
9 #define      ILI9806G_CS_PORT             GPIOG
10 #define      ILI9806G_CS_PIN              GPIO_Pin_10
11 #define      ILI9806G_CS_PinSource       GPIO_PinSource10
12
13 //DC 引脚, 使用 FSMC 的地址信号控制, 本引脚决定了访问 LCD 时使用的地址
14 //PF0 为 FSMC_A0
15 #define      ILI9806G_DC_CLK              RCC_AHB1Periph_GPIOF
16 #define      ILI9806G_DC_PORT             GPIOF
17 #define      ILI9806G_DC_PIN              GPIO_Pin_0
18 #define      ILI9806G_DC_PinSource       GPIO_PinSource0
```



```
19
20 //写使能
21 #define ILI9806G_WR_CLK RCC_AHB1Periph_GPIOD
22 #define ILI9806G_WR_PORT GPIOD
23 #define ILI9806G_WR_PIN GPIO_Pin_5
24 #define ILI9806G_WR_PinSource GPIO_PinSource5
25
26 //读使能
27 #define ILI9806G_RD_CLK RCC_AHB1Periph_GPIOD
28 #define ILI9806G_RD_PORT GPIOD
29 #define ILI9806G_RD_PIN GPIO_Pin_4
30 #define ILI9806G_RD_PinSource GPIO_PinSource4
31
32 //复位引脚
33 #define ILI9806G_RST_CLK RCC_AHB1Periph_GPIOF
34 #define ILI9806G_RST_PORT GPIOF
35 #define ILI9806G_RST_PIN GPIO_Pin_11
36
37 //背光引脚
38 #define ILI9806G_BK_CLK RCC_AHB1Periph_GPIOF
39 #define ILI9806G_BK_PORT GPIOF
40 #define ILI9806G_BK_PIN GPIO_Pin_9
41
42 /*****数据信号线*****/
43 #define ILI9806G_D0_CLK RCC_AHB1Periph_GPIOD
44 #define ILI9806G_D0_PORT GPIOD
45 #define ILI9806G_D0_PIN GPIO_Pin_14
46 #define ILI9806G_D0_PinSource GPIO_PinSource14
47 /*****此处省略其它数据线*****/
```

以上代码根据硬件的连接，把与 FSMC 与液晶屏通讯使用的引脚号、引脚源以及复用功能映射都以宏封装起来。其中请着重关注代码中液晶屏 LCD_CS 及 LCD_RS(DC 引脚)与 FSMC 存储区选择引脚 FSMC_NE 及地址信号 FSMC_A 的编号，它们会决定 STM32 要使用什么内存地址来控制与液晶屏的通讯。

初始化 FSMC 的 GPIO

利用上面的宏，编写 FSMC 的 GPIO 引脚初始化函数，见代码清单 28-4。

代码清单 28-4 FSMC 的 GPIO 初始化函数(省略了部分数据线)

```
1
2 /**
3  * @brief  初始化 ILI9806G 的 IO 引脚
4  * @param  无
5  * @retval 无
6  */
7 static void ILI9806G_GPIO_Config ( void )
8 {
9     GPIO_InitTypeDef GPIO_InitStructure;
10
11     /* 使能 FSMC 对应相应管脚时钟 */
12     RCC_AHB1PeriphClockCmd (
13         /*控制信号*/
14         ILI9806G_CS_CLK|ILI9806G_DC_CLK|ILI9806G_WR_CLK|
15         ILI9806G_RD_CLK |ILI9806G_BK_CLK|ILI9806G_RST_CLK|
16         /*数据信号*/
17         ILI9806G_D0_CLK|, ENABLE );
18
19
20     /* 配置 FSMC 相对应的数据线,FSMC-D0~D15 */
21     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
22     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
23     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
```

```
24     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;
25
26     GPIO_InitStructure.GPIO_Pin = ILI9806G_D0_PIN;
27     GPIO_Init(ILI9806G_D0_PORT, &GPIO_InitStructure);
28     GPIO_PinAFConfig(ILI9806G_D0_PORT, ILI9806G_D0_PinSource, FSMC_AF);
29
30     /* 配置 FSMC 相对应的控制线
31     *   FSMC_NOE      :LCD-RD
32     *   FSMC_NWE      :LCD-WR
33     *   FSMC_NE1      :LCD-CS
34     *   FSMC_A0       :LCD-DC
35     */
36     GPIO_InitStructure.GPIO_Pin = ILI9806G_RD_PIN;
37     GPIO_Init(ILI9806G_RD_PORT, &GPIO_InitStructure);
38     GPIO_PinAFConfig(ILI9806G_RD_PORT, ILI9806G_RD_PinSource, FSMC_AF);
39
40     GPIO_InitStructure.GPIO_Pin = ILI9806G_WR_PIN;
41     GPIO_Init(ILI9806G_WR_PORT, &GPIO_InitStructure);
42     GPIO_PinAFConfig(ILI9806G_WR_PORT, ILI9806G_WR_PinSource, FSMC_AF);
43
44     GPIO_InitStructure.GPIO_Pin = ILI9806G_CS_PIN;
45     GPIO_Init(ILI9806G_CS_PORT, &GPIO_InitStructure);
46     GPIO_PinAFConfig(ILI9806G_CS_PORT, ILI9806G_CS_PinSource, FSMC_AF);
47
48     GPIO_InitStructure.GPIO_Pin = ILI9806G_DC_PIN;
49     GPIO_Init(ILI9806G_DC_PORT, &GPIO_InitStructure);
50     GPIO_PinAFConfig(ILI9806G_DC_PORT, ILI9806G_DC_PinSource, FSMC_AF);
51
52     /* 配置 LCD 复位 RST 控制管脚*/
53     GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
54     GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
55     GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
56     GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
57
58     GPIO_InitStructure.GPIO_Pin = ILI9806G_RST_PIN;
59     GPIO_Init ( ILI9806G_RST_PORT, & GPIO_InitStructure );
60
61     /* 配置 LCD 背光控制管脚 BK*/
62     GPIO_InitStructure.GPIO_Pin = ILI9806G_BK_PIN;
63     GPIO_Init ( ILI9806G_BK_PORT, & GPIO_InitStructure );
64 }
```

与控制 SRAM 中的 GPIO 初始化类似，对于 FSMC 引脚，全部直接初始化为复用推挽输出模式即可，而背光 BK 引脚及液晶复信 RST 信号则被初始化成普通的推挽输出模式，这两个液晶控制信号直接输出普通的电平控制即可。

配置 FSMC 的模式

接下来需要配置 FSMC 的工作模式，见代码清单 28-5。

代码清单 28-5 配置 FSMC 的模式

```
1 //由片选引脚决定的 NOR/SRAM 块
2 #define      FSMC_Bank1_NORSRAMx          FSMC_Bank1_NORSRAM3
3
4 /***** ILI9806G 显示屏 8080 通讯引脚定义 *****/
5 /*****控制信号线*****/
6 #define      FSMC_AF                      GPIO_AF_FSMC
7 //片选，选择 NOR/SRAM 块
8 #define      ILI9806G_CS_CLK              RCC_AHB1Periph_GPIOG
9 #define      ILI9806G_CS_PORT             GPIOG
10 #define      ILI9806G_CS_PIN              GPIO_Pin_10
11 #define      ILI9806G_CS_PinSource        GPIO_PinSource10
12 /*****
13 **
14 * @brief LCD FSMC 模式配置
```

```
15 * @param 无
16 * @retval 无
17 */
18 static void ILI9806G_FSMC_Config ( void )
19 {
20     FSMC_NORSRAMInitTypeDef FSMC_NORSRAMInitStructure;
21     FSMC_NORSRAMTimingInitTypeDef readWriteTiming;
22
23     /* 使能 FSMC 时钟 */
24     RCC_AHB3PeriphClockCmd(RCC_AHB3Periph_FSMC, ENABLE);
25
26     //地址建立时间 (ADDSET) 为 1 个 HCLK 5/168M=30ns
27     readWriteTiming.FSMC_AddressSetupTime = 0x04; //地址建立时间
28     //数据保持时间 (DATAST) + 1 个 HCLK = 12/168M=72ns
29     readWriteTiming.FSMC_DataSetupTime = 0x0b; //数据建立时间
30     //选择控制的模式
31     //模式 B, 异步 NOR FLASH 模式, 与 ILI9806G 的 8080 时序匹配
32     readWriteTiming.FSMC_AccessMode = FSMC_AccessMode_B;
33
34     /*以下配置与模式 B 无关*/
35     //地址保持时间 (ADDHLD) 模式 A 未用到
36     readWriteTiming.FSMC_AddressHoldTime = 0x00; //地址保持时间
37     //设置总线转换周期, 仅用于复用模式的 NOR 操作
38     readWriteTiming.FSMC_BusTurnAroundDuration = 0x00;
39     //设置时钟分频, 仅用于同步类型的存储器
40     readWriteTiming.FSMC_CLKDivision = 0x00;
41     //数据保持时间, 仅用于同步型的 NOR
42     readWriteTiming.FSMC_DataLatency = 0x00;
43
44     FSMC_NORSRAMInitStructure.FSMC_Bank = FSMC_Bank1_NORSRAMx;
45     FSMC_NORSRAMInitStructure.FSMC_DataAddressMux = FSMC_DataAddressMux_Disable;
46     FSMC_NORSRAMInitStructure.FSMC_MemoryType = FSMC_MemoryType_NOR;
47     FSMC_NORSRAMInitStructure.FSMC_MemoryDataWidth = FSMC_MemoryDataWidth_16b;
48     FSMC_NORSRAMInitStructure.FSMC_BurstAccessMode = FSMC_BurstAccessMode_Disable;
49     FSMC_NORSRAMInitStructure.FSMC_WaitSignalPolarity = FSMC_WaitSignalPolarity_Low;
50     FSMC_NORSRAMInitStructure.FSMC_WrapMode = FSMC_WrapMode_Disable;
51     FSMC_NORSRAMInitStructure.FSMC_WaitSignalActive = FSMC_WaitSignalActive_BeforeWaitState;
52     FSMC_NORSRAMInitStructure.FSMC_WriteOperation = FSMC_WriteOperation_Enable;
53     FSMC_NORSRAMInitStructure.FSMC_WaitSignal = FSMC_WaitSignal_Disable;
54     FSMC_NORSRAMInitStructure.FSMC_ExtendedMode = FSMC_ExtendedMode_Disable;
55     FSMC_NORSRAMInitStructure.FSMC_WriteBurst = FSMC_WriteBurst_Disable;
56     FSMC_NORSRAMInitStructure.FSMC_ReadWriteTimingStruct = &readWriteTiming;
57     FSMC_NORSRAMInitStructure.FSMC_WriteTimingStruct = &readWriteTiming;
58
59     FSMC_NORSRAMInit ( & FSMC_NORSRAMInitStructure );
60
61     /* 使能 FSMC_Bank1_NORSRAM3 */
62     FSMC_NORSRAMCmd ( FSMC_Bank1_NORSRAMx, ENABLE );
63 }
```

这个函数的主体是把 FSMC 配置成异步 NOR FLASH 使用的模式 B, 使用该方式模拟 8080 时序控制液晶屏, 执行流程如下:

(1) 初始化 FSMC 时钟

函数开头使用库函数 `RCC_AHB3PeriphClockCmd` 使能 FSMC 外设的时钟。

(2) 时序结构体赋值

接下来对时序结构体 `FSMC_NORSRAMTimingInitTypeDef` 赋值。在这个时序结构体配置中, 由于我们要使用异步 NOR FLASH 的方式模拟 8080 时序, 所以选择 FSMC 为模式 B, 在该模式下配置 FSMC 的控制时序结构体中, 实际上只有地址建立时间

`FSMC_AddressSetupTime` (即 `ADDSET` 的值) 以及数据建立时间 `FSMC_DataSetupTime`

（即 DATAST 的值）成员的配置值是有效的，其它异步 NOR FLASH 没使用到的成员值全配置为 0 即可。而且，这些成员值使用的单位为：1 个 HCLK 的时钟周期，而 HCLK 的时钟频率为 168MHz，对应每个时钟周期为 1/168 微秒。

由图 28-23 及图 28-24 中的 ILI9806G 时序参数说明及要求可大致得知 ILI9806G 的写周期为最小 $t_{wc} = 30ns$ ，而读周期最小为 $t_{rdl}+t_{rod}=45+20=65ns$ 。（对于读周期表中有参数要一个要求为 trcfm 和 trc 分别为 450ns 及 160ns，但经过测试并不需要遵照它们的指标要求）

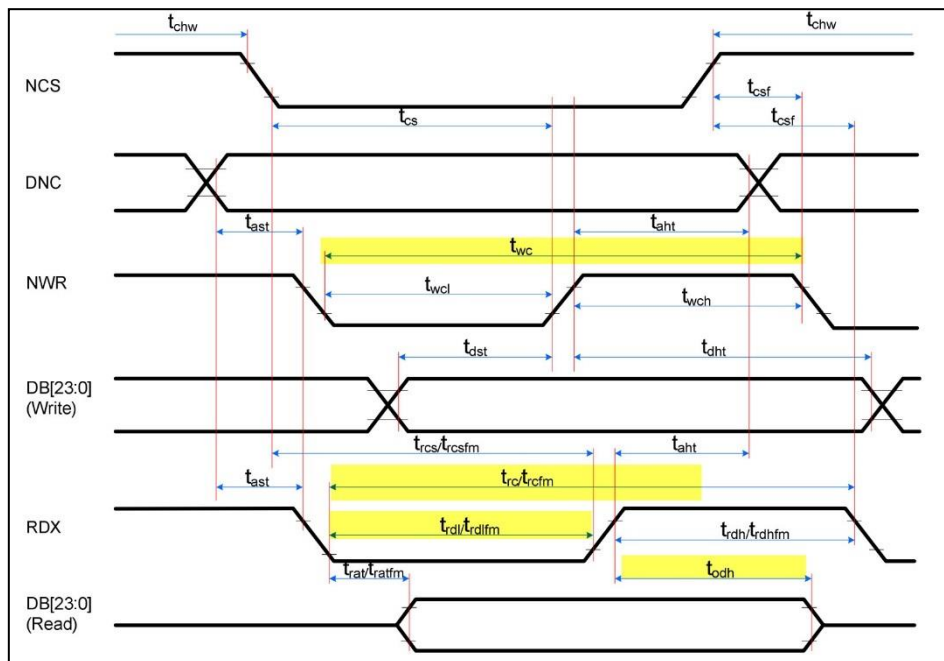


图 28-23 ILI9806G 时序参数说明图(摘自 ILI9806G 数据手册)

Signal	Symbol	Parameter	min	max	Unit	Description
DCX	tast	Address setup time	0	-	ns	-
	taht	Address hold time (Write/Read)	10	-	ns	-
CSX	tchw	CSX "H" pulse width	0	-	ns	-
	tcs	Chip Select setup time (Write)	10	-	ns	-
	trcs	Chip Select setup time (Read ID)	45	-	ns	-
	trcsfm	Chip Select setup time (Read FM)	355	-	ns	-
	tcsf	Chip Select Wait time (Write/Read)	10	-	ns	-
WRX	twc	Write cycle	30	-	ns	-
	twrh	Write Control pulse H duration	10	-	ns	-
	twrl	Write Control pulse L duration	10	-	ns	-
RDX (FM)	trcfm	Read Cycle (FM)	450	-	ns	When read from the Frame Memory
	trdhfm	Read Control H duration (FM)	90	-	ns	
	trdlfm	Read Control L duration (FM)	355	-	ns	
RDX (ID)	trc	Read cycle (ID)	160	-	ns	When read ID data
	trdh	Read Control pulse H duration	90	-	ns	
	trdl	Read Control pulse L duration	45	-	ns	
DB[17:0], DB[15:0], DB[8:0] DB[7:0]	tdst	Write data setup time	10	-	ns	CL = 30pF (maximum) CL = 8pF (minimum)
	tdht	Write data hold time	10	-	ns	
	trat	Read access time	-	40	ns	
	tratfm	Read access time	-	340	ns	
	trodh	Read output disable time	20	80	ns	

图 28-24 ILI9806G 的时序参数要求(摘自 ILI9806G 数据手册)

在 FSMC 代码中使用结构体中的 FSMC_AddressSetupTime（即 ADDSET 的值）及 FSMC_DataSetupTime（即 DATAST 的值）成员控制 FSMC 的读写周期，见图 28-25。

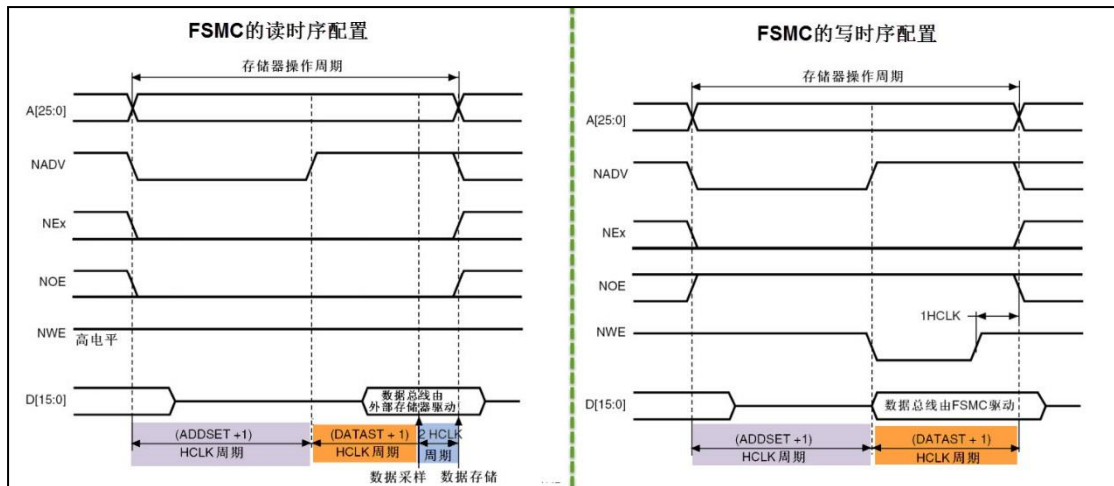


图 28-25FSMC 的读写时序

结合 ILI9806G 的时序要求和 FSMC 的配置图，代码中按照读写时序周期均要求至少 30ns 来计算，配置结果为 ADDSET = 1 及 DATST = 4，把时间单位 1/168 微秒(即 1000/168 纳秒)代入，因此读写周期的时间被配置为：

读周期： $t_{rc} = ((ADDSET+1) + (DATST+1) + 2) * 1000/168 = ((4+1) + (4+1) + 2) * 1000/168 = 71ns$

写周期： $t_{wc} = ((ADDSET+1) + (DATST+1)) * 1000/168 = ((1+4) + (4+1)) * 1000/168 = 60ns$

所以把这两个参数值写入到 FSMC 后，它控制的读写周期都比 ILI9806G 的最低要求值大。（经测试，这两个参数值也可以适当减小，您可以亲自试一下）

把计算得的参数赋值到时序结构体中的 FSMC_AddressSetupTime（即 ADDSET 的值）及 FSMC_DataSetupTime（即 DATST 的值）中，然后再把时序结构体作为指针赋值到下面的 FSMC 初始化结构体中，作为读写的时序参数，最后再调用 FSMC_NORSRAMInit 函数即可把参数写入到相应的寄存器中。

(3) 配置 FSMC 初始化结构体

函数接下来对 FSMC SRAM 的初始化结构体赋值。主要包括存储映射区域、存储器类型以及数据线宽度等，这些是根据外部电路设置的。

❑ 设置存储区域 FSMC_Bank

FSMC_Bank 成员设置 FSMC 的 NOR FLASH 存储区域映射选择为宏

FSMC_Bank1_NORSRAMx (即 FSMC_Bank1_NORSRAM3)，这是由于我们的 SRAM 硬件连接到 FSMC_NE3 和 NOR/PSRAM 相关引脚，所以对应到存储区域 Bank1 SRAM3，对应的基地址为 0X6800 0000；

❑ 存储器类型 FSMC_MemoryType

由于使用异步 NOR FLASH 模式模拟 8080 时序，所以 FSMC_MemoryType 成员要选择相应的 FSMC_MemoryType_NOR；

❑ 数据线宽度 FSMC_MemoryDataWidth

根据硬件的数据线连接，数据线宽度被配置为 16 位宽

FSMC_MemoryDataWidth_16b；

❑ 写使能 FSMC_WriteOperation

FSMC_WriteOperation 用于设置写使能，只有使能了才能正常使用 FSMC 向外部存储器写入数据；

❑ 扩展模式以及读写时序

在 FSMC_ExtendedMode 成员中可以配置是否使用扩展模式，当设置扩展模式时，读时序使用 FSMC_ReadWriteTimingStruct 中的配置，写时序使用 FSMC_WriteTimingStruct 中的配置，两种配置互相独立，可以赋值为不同的读写时序结构体。在本实例中不使用扩展模式，即读写时序使用相同的配置，都是赋值为前面的 readWriteTiming 结构体；

❑ 其它

配置 FSMC 还涉及到其它的结构体成员，但这些结构体成员与异步 NOR FLASH 控制不相关，都被设置为 Disable 了；

赋值完成后调用库函数 FSMC_NORSRAMInit 把初始化结构体配置的各种参数写入到 FSMC_BCR 控制寄存器及 FSMC_BTR 时序寄存器中。最后调用 FSMC_NORSRAMCmd 函数使能要控制的存储区域 FSMC_Bank1_NORSRAM3。

计算控制液晶屏时使用的地址

初始化完 FSMC 后，即可使用类似扩展外部 SRAM 中的读取方式：通过访问某个地址，由 FSMC 产生时序与外部存储器通讯，进行读写。

同样地，当访问特定地址时，FSMC 会产生相应的模拟 8080 时序，控制地址线输出要访问的内存地址，使用数据信号线接收或发送数据，其它片选信号 NE、读使能信号 NOE、写使能信号 NWE 辅助产生完整的时序，而由于控制液晶屏的硬件连接中，使用如图 28-26 中的连接来模拟 8080 时序，所以 FSMC 产生的这些信号会被 ILI9806G 接收，并且使用其中一根 FSMC_Ax 地址控制命令/数据选择引脚 RS(即 D/CX)，因此，需要重点理解下当 STM32 访问什么地址时，对应的 FSMC_Ax 引脚会输出高电平表示传输的是数据，访问什么地址时，对应的 FSMC_Ax 引脚会输出低电平表示传输的是命令。若理解了计算过程，以后就可以根据自己制作的硬件电路来计算访问地址了。

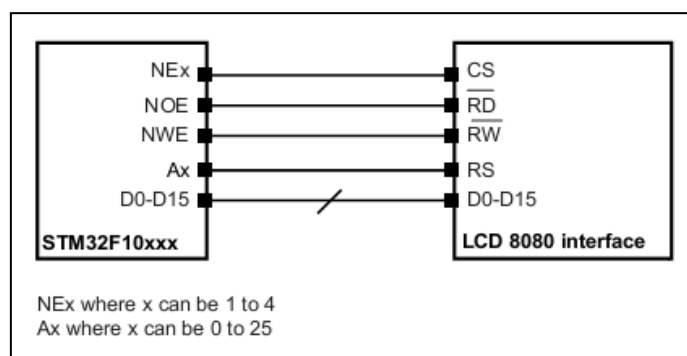


图 28-26 FSMC 与 8080 端口连接简图

计算地址的过程如下：

- (1) 本工程中使用的是 FSMC_NE3 作为 8080_CS 片选信号，所以首先可以确认地址范围，当访问 0X6800 0000 ~ 0X6BFF FFFF 地址时，FSMC 均会对外产生片选有效的访问时序；
- (2) 本工程中使用 FSMC_A0 地址线作为命令/数据选择线 RS 信号，所以在以上地址范围内，再选择出使得 FSMC_A0 输出高电平的地址，即可控制表示数据，选择出使得 FSMC_A0 输出低电平的地址，即可控制表示命令。
- ❑ 要使 FSMC_A0 地址线为高电平，实质是输出地址信号的第 0 位为 1 即可，使用 0X6800 0000~0X6BFF FFFF 内的任意地址，作如下运算：
设置地址的第 0 位为 1: $0X6800\ 0000 \mid= (1 << 0) = 0x6800\ 0001$
- ❑ 要使 FSMC_A0 地址线为低电平，实质是输出地址信号的第 0 位为 0 即可，使用 0X6800 0000~0X6BFF FFFF 内的任意地址，作如下运算：
设置地址的第 0 位为 0: $0X6800\ 0000 \&= \sim (1 << 0) = 0x6800\ 0000$
- (3) 但是，以上方法计算的地址还不完全正确，根据《STM32 参考手册》对 FSMC 访问 NOR FLASH 的说明，见图 28-27，STM32 内部访问地址时使用的是内部 HADDR 总线，它是需要转换到外部存储器的内部 AHB 地址线，它是字节地址(8 位)，而存储器访问不都是按字节访问，因此接到存储器的地址线依存储器的数据宽度有所不同。

数据宽度 ⁽¹⁾	连到存储器的地址线	最大访问存储器空间(位)
8位	HADDR[25:0]与FSMC_A[25:0]对应相连	64M字节 x 8 = 512 M位
16位	HADDR[25:1]与FSMC_A[24:0]对应相连，HADDR[0]未接	64M字节/2 x 16 = 512 M位

(1) 对于16位宽度的外部存储器，FSMC将在内部使用HADDR[25:1]产生外部存储器的地址FSMC_A[24:0]。不论外部存储器的宽度是多少(16位或8位)，FSMC_A[0]始终应该连到外部存储器的地址线A[0]。

图 28-27 《STM32 参考手册》中对 HADDR 与 FSMC 地址线的说明

在本工程中使用的是 16 位的数据访问方式，所以 HADDR 与 FSMC_A 的地址线连接关系会左移一位，如 HADDR1 与 FSMC_A0 对应、HADDR2 与 FSMC_A1 对应。因此，当 FSMC_A0 地址线为 1 时，实际上内部地址的第 1 位为 1，FSMC_A1 地址线为 1 时，实际上内部地址的第 2 位为 1。同样地，当希望 FSMC_A0 地址输出高电平或低电平时，需要重新调整计算公式：

- ❑ 要使 FSMC_A0 地址线为高电平，实质是访问内部 HADDR 地址的第(0+1)位为 1 即可，使用 0X6800 0000~0X6BFF FFFF 内的任意地址，作如下运算：
使 FSMC_A0 地址线为高电平: $0X6800\ 0000 \mid= (1 << (0+1)) = 0x6800\ 0002$
- ❑ 要使 FSMC_A0 地址线为低电平，实质是访问内部 HADDR 地址的第(0+1)位为 0 即可，使用 0X6800 0000~0X6BFF FFFF 内的任意地址，作如下运算：
使 FSMC_A0 地址线为低电平: $0X6800\ 0000 \&= \sim (1 << (0+1)) = 0x6800\ 0000$

根据最终的计算结果，总结如下：当 STM32 访问内部的 0x6800 0002 地址时，FSMC 自动输出时序，且使得与液晶屏的数据/命令选择线 RS(即 D/CX)相连的 FSMC_A0 输出高电平，使得液晶屏会把传输过程理解为数据传输；类似地，当 STM32 访问内部的 0X6800

0000 地址时, FSMC 自动输出时序, 且使得与液晶屏的数据/命令选择线 RS(即 D/CX)相连的 FSMC_A0 输出低电平, 使得液晶屏会把传输过程理解为命令传输。

在工程代码中, 把以上计算结果封装成了宏, 见代码清单 28-6。

代码清单 28-6 使用 FSMC 访问数据及访问命令的地址 (bsp_ILI9806G_lcd.h 文件)

```
1 /*****
2 2^26=0X0400 0000 = 64MB,每个 BANK 有 4*64MB = 256MB
3 64MB:FSMC_Bank1_NORSRAM1:0X6000 0000 ~ 0X63FF FFFF
4 64MB:FSMC_Bank1_NORSRAM2:0X6400 0000 ~ 0X67FF FFFF
5 64MB:FSMC_Bank1_NORSRAM3:0X6800 0000 ~ 0X6BFF FFFF
6 64MB:FSMC_Bank1_NORSRAM4:0X6C00 0000 ~ 0X6FFF FFFF
7
8 选择 BANK1-BORSRAM3 连接 TFT, 地址范围为 0X6800 0000 ~ 0X6BFF FFFF
9 FSMC_A0 接 LCD 的 DC(寄存器/数据选择)脚
10 寄存器基地址 = 0X6C00 0000
11 RAM 基地址 = 0X6D00 0000 = 0X6C00 0000+2^0*2 = 0X6800 0000 + 0X2 = 0X6800 0002
12 当选择不同的地址线时, 地址要重新计算
13 *****/
14
15 /***** ILI9806G 显示屏的 FSMC 参数定义 *****/
16 //FSMC_Bank1_NORSRAM 用于 LCD 命令操作的地址
17 #define FSMC_Addr_ILI9806G_CMD ( ( uint32_t ) 0x68000000 )
18
19 //FSMC_Bank1_NORSRAM 用于 LCD 数据操作的地址
20 #define FSMC_Addr_ILI9806G_DATA ( ( uint32_t ) 0x68000002 )
```

利用这样的宏, 再使用指针的形式访问其地址, 即可控制 FSMC 产生相应的时序, 工程代码中把发送命令及发送数据的操作封装成了内联函数, 方便后面调用, 见代码清单 28-7。

代码清单 28-7 向液晶屏发送命令及发送数据的操作

```
1 /**
2  * @brief 向 ILI9806G 写入命令
3  * @param usCmd :要写入的命令(表寄存器地址)
4  * @retval 无
5  */
6 __inline void ILI9806G_Write_Cmd ( uint16_t usCmd )
7 {
8     * ( __IO uint16_t * ) ( FSMC_Addr_ILI9806G_CMD ) = usCmd;
9 }
10
11
12
13 /**
14  * @brief 向 ILI9806G 写入数据
15  * @param usData :要写入的数据
16  * @retval 无
17  */
18 __inline void ILI9806G_Write_Data ( uint16_t usData )
19 {
20     * ( __IO uint16_t * ) ( FSMC_Addr_ILI9806G_DATA ) = usData;
21 }
22
23
```

需要写操作时, 只要把要发送的命令代码或数据作为参数输入到函数然后调用即可, 对于液晶屏的读操作, 把向指针赋值的过程改为读取指针内容即可。

向液晶屏写入初始化配置

利用上面的发送命令及数据操作，可以向液晶屏写入一些初始化配置，见代码清单 28-8。

代码清单 28-8 向液晶屏写入初始化配置

```
1 /**
2  * @brief  初始化 ILI9806G 寄存器
3  * @param  无
4  * @retval 无
5  */
6 static void ILI9806G_REG_Config ( void )
7 {
8     /* EXTC Command Set enable register */
9     DEBUG_DELAY ();
10    ILI9806G_Write_Cmd ( 0xFF );
11    ILI9806G_Write_Data ( 0xFF );
12    ILI9806G_Write_Data ( 0x98 );
13    ILI9806G_Write_Data ( 0x06 );
14
15    /* GIP 1(BCh) */
16    DEBUG_DELAY ();
17    ILI9806G_Write_Cmd(0xBC);
18    ILI9806G_Write_Data(0x01);
19    ILI9806G_Write_Data(0x0E);
20    ILI9806G_Write_Data(0x61);
21    ILI9806G_Write_Data(0xFB);
22    ILI9806G_Write_Data(0x10);
23    ILI9806G_Write_Data(0x10);
24    ILI9806G_Write_Data(0x0B);
25    ILI9806G_Write_Data(0x0F);
26    ILI9806G_Write_Data(0x2E);
27    ILI9806G_Write_Data(0x73);
28    ILI9806G_Write_Data(0xFF);
29    /*以下省略大量配置内容*/
30 }
```

以上列出的代码中省略了大量的配置内容，本质上它们都是使用 ILI9806G_Write_Cmd 发送代码，然后使用 ILI9806G_Write_Data 函数发送命令对应的参数对液晶屏进行配置。

这个初始化过程中发送的代码及参数主要是配置了液晶屏的上电过程、显示屏的伽玛参数、分辨率、像素格式等内容，这些配置主要由液晶屏生产厂家提供，本教程后面只针对常用命令进行讲解，此处不作详细说明，关于命令及参数可以查询《ILI9806G-Data Sheet》获知，在该文档中搜索命令代码即可方便定位到相应的说明。例如，要查找代码中的 0xCF 命令说明，在文档中搜索“CFh”即可，见图 28-28。

ILI9806G-Data Sheet... x

Translat



a-Si TFT LCD Single Chip Driver
480 (RGB) x 864 Resolution and 16.7M-color

ILI9806G

4.3.42. EXTC Command Set enable register (FFh)

FFh	ENEXTC (EXTC command set enable register)												
	DCX	RDX	WRX	D [23:8]	D7	D6	D5	D4	D3	D2	D1	D0	HEX
Command	0	1	↑	XX	1	1	1	1	1	1	1	1	FFh
1 st Parameter	1	1	↑	XX	1	1	1	1	1	1	1	1	FFh
2 nd Parameter	1	1	↑	XX	1	0	0	1	1	0	0	0	98h
3 rd Parameter	1	1	↑	XX	0	0	0	0	0	1	1	0	06h
Description	Set the register, 1 st Parameter = FFh, 2 nd Parameter = 98h, 3 rd Parameter = 06h to enable "EXTC command set". Set the register, 1 st Parameter = FFh, 2 nd Parameter = 98h, 3 rd Parameter = any value except 06h to disable "EXTC command set".												
Restriction													

图 28-28 在《ILI9806G 数据手册》中 CFh 命令的部分说明

设置液晶显示窗口

根据液晶屏的要求，在发送显示数据前，需要先设置显示窗口确定后面发送的像素数据的显示区域，见代码清单 28-9。

代码清单 28-9 设置液晶显示窗口

```

1 /***** ILI9806 命令 *****/
2 #define      CMD_SetCoordinateX      0x2A      //设置 x 坐标
3 #define      CMD_SetCoordinateY      0x2B      //设置 y 坐标
4
5 /**
6  * @brief 在 ILI9806G 显示器上开辟一个窗口
7  * @param usX : 在特定扫描方向下窗口的起点 x 坐标
8  * @param usY : 在特定扫描方向下窗口的起点 y 坐标
9  * @param usWidth : 窗口的宽度
10 * @param usHeight : 窗口的高度
11 * @retval 无
12 */
13 void ILI9806G_OpenWindow ( uint16_t usX, uint16_t usY,
                             uint16_t usWidth, uint16_t usHeight )
14 {
15     ILI9806G_Write_Cmd ( CMD_SetCoordinateX );          /* 设置 x 坐标 */
16     ILI9806G_Write_Data ( usX >> 8 ); /* 先高 8 位, 然后低 8 位 */
17     ILI9806G_Write_Data ( usX & 0xff ); /* 设置起始点和结束点 */
18     ILI9806G_Write_Data ( ( usX + usWidth - 1 ) >> 8 );
19     ILI9806G_Write_Data ( ( usX + usWidth - 1 ) & 0xff );
20
21     ILI9806G_Write_Cmd ( CMD_SetCoordinateY );          /* 设置 y 坐标 */
22     ILI9806G_Write_Data ( usY >> 8 );
23     ILI9806G_Write_Data ( usY & 0xff );
24     ILI9806G_Write_Data ( ( usY + usHeight - 1 ) >> 8 );
25     ILI9806G_Write_Data ( ( usY + usHeight - 1 ) & 0xff );
26 }

```

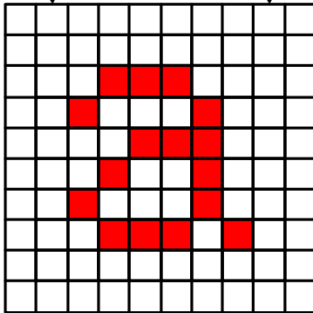

2Ah	CASET (Column Address Set)												
	D/CX	RDX	WRX	D17-8	D7	D6	D5	D4	D3	D2	D1	D0	HEX
Command	0	1	↑	XX	0	0	1	0	1	0	1	0	2Ah
1 st Parameter	1	1	↑	XX	SC15	SC14	SC13	SC12	SC11	SC10	SC9	SC8	Note1
2 nd Parameter	1	1	↑	XX	SC7	SC6	SC5	SC4	SC3	SC2	SC1	SC0	
3 rd Parameter	1	1	↑	XX	EC15	EC14	EC13	EC12	EC11	EC10	EC9	EC8	Note1
4 th Parameter	1	1	↑	XX	EC7	EC6	EC5	EC4	EC3	EC2	EC1	EC0	
Description	This command is used to define area of frame memory where MCU can access. This command makes no change on the other driver status. The values of SC [15:0] and EC [15:0] are referred when RAMWR command comes. Each value represents one column line in the Frame Memory.												
	SC[15:0] EC[15:0]  X = Don't care												

图 28-29 设置显示窗口的 X 坐标(2Ah 命令) (摘自《ILI9806G 数据手册》)

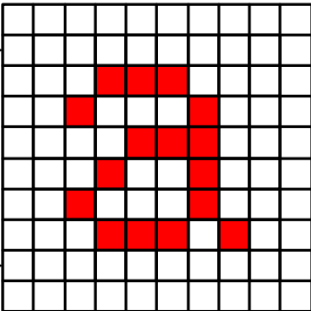
2Bh	PASET (Page Address Set)													
	D/CX	RDX	WRX	D17-8	D7	D6	D5	D4	D3	D2	D1	D0	HEX	
Command	0	1	↑	XX	0	0	1	0	1	0	1	1	2Bh	
1 st Parameter	1	1	↑	XX	SP15	SP14	SP13	SP12	SP11	SP10	SP9	SP8	Note1	
2 nd Parameter	1	1	↑	XX	SP7	SP6	SP5	SP4	SP3	SP2	SP1	SP0		
3 rd Parameter	1	1	↑	XX	EP15	EP14	EP13	EP12	EP11	EP10	EP9	EP8	Note1	
4 th Parameter	1	1	↑	XX	EP7	EP6	EP5	EP4	EP3	EP2	EP1	EP0		
Description	This command is used to define area of frame memory where MCU can access. This command makes no change on the other driver status. The values of SP [15:0] and EP [15:0] are referred when RAMWR command comes. Each value represents one Page line in the Frame Memory.													
	SP[15:0] →  EP[15:0] → X = Don't care													

图 28-30 设置液晶显示窗口的 Y 坐标 (2Bh 命令) (摘自《ILI9806G 数据手册》)

代码中定义的 ILI9806G_OpenWindow 函数实现了图 28-29 及图 28-30 中的 0x2A 和 0x2B 命令，它们分别用于设置显示窗口的起始及结束的 X 坐标和 Y 坐标，每个命令后包含 4 个 8 位的参数，这些参数组合后成起始坐标和结束坐标各 1 个用 16 位表示的值。

ILI9806G_OpenWindow 把它的四个函数输入参数 X、Y 起始坐标，宽度、高度转化成命令参数的格式，写入到液晶屏中，从而设置出一个显示窗口。

发送像素数据

调用上面的 ILI9806G_OpenWindow 函数设置显示窗口后，再向液晶屏发送像素数据时，这些数据就会直接显示在它设定的窗口位置中。发送像素数据的操作见代码清单 28-10。

代码清单 28-10 发送像素数据

```

1
2 #define          CMD_SetPixel          0x2C          //填充像素
3
4 /**
5  * @brief 在 ILI9806G 显示器上以某一颜色填充像素点
6  * @param ulAmout_Point : 要填充颜色的像素点的总数目
7  * @param usColor : 颜色
8  * @retval 无
9  */
10 static __inline void ILI9806G_FillColor ( uint32_t ulAmout_Point,
11                                           uint16_t usColor )
12 {
13     uint32_t i = 0;
14     /* memory write */
15     ILI9806G_Write_Cmd ( CMD_SetPixel );
16
17     for ( i = 0; i < ulAmout_Point; i ++ )
18         ILI9806G_Write_Data ( usColor );
19 }

```

2Ch	RAMWR (Memory Write)												
	D/CX	RDX	WRX	D17-8	D7	D6	D5	D4	D3	D2	D1	D0	HEX
Command	0	1	↑	XX	0	0	1	0	1	1	0	0	2Ch
1 st Parameter	1	1	↑	D1 [17:0]									XX
:	1	1	↑	Dx [17:0]									XX
N th Parameter	1	1	↑	Dn [17:0]									XX
Description	This command is used to transfer data from MCU to frame memory. This command makes no change to the other driver status. When this command is accepted, the column register and the page register are reset to the Start Column/Start Page positions. The Start Column/Start Page positions are different in accordance with MADCTL setting.) Then D [17:0] is stored in frame memory and the column register and the page register incremented. Sending any other command can stop frame Write. X = Don't care.												

图 28-31 发送像素数据（2Ch 命令）（摘自《ILI9806G 数据手册》）

发送像素数据的命令非常简单，首先发送命令代码 0x2C，然后后面紧跟着要传输的像素数据即可。按照本液晶屏的配置，像素点的格式为 RGB565，所以像素数据就是要显示的 RGB565 格式的颜色值。

本 ILI9806G_FillColor 函数包含两个输入参数，分别用于设置要发送的像素数据个数 ulAmout_Point 及像素点的颜色值 usColor，在代码实现中它调用 ILI9806G_Write_Cmd 发送一次命令代码，接着使用 for 循环调用 ILI9806G_Write_Data 写入 ulAmout_Point 个同样的颜色值。

这些颜色值会按顺序填充到前面使用 ILI9806G_OpenWindow 函数设置的显示窗口中，例如，若设置了一个 usX=10, usY=30, usWidth=50, usHeight=20 的窗口，然后再连续填

充 50*20 个颜色值为 0XFFFF 的像素数据，即可在(10,30)的起始坐标处显示一个宽 50 像素高 20 像素的白色矩形。

绘制单个像素点

利用前面的 ILI9806G_OpenWindow 和 ILI9806G_FillColor 函数，可以正式开始控制液晶屏绘制特定的图像，而所有图像都是由多个像素点组成的，单个像素点的绘制函数见代码清单 28-11。

代码清单 28-11 绘制单个像素点

```
1
2 static uint16_t CurrentTextColor    = BLACK;//前景色
3 static uint16_t CurrentBackColor    = WHITE;//背景色
4
5 /**
6  * @brief  设定 ILI9806G 的光标坐标
7  * @param  usX : 在特定扫描方向下光标的 x 坐标
8  * @param  usY : 在特定扫描方向下光标的 y 坐标
9  * @retval 无
10 */
11 static void ILI9806G_SetCursor ( uint16_t usX, uint16_t usY )
12 {
13     ILI9806G_OpenWindow ( usX, usY, 1, 1 );
14 }
15
16
17 /**
18  * @brief  对 ILI9806G 显示器的某一点以某种颜色进行填充
19  * @param  usX : 在特定扫描方向下该点的 x 坐标
20  * @param  usY : 在特定扫描方向下该点的 y 坐标
21  * @note   可使用 LCD_SetBackColor、LCD_SetTextColor、LCD_SetColors 函数设置颜色
22  * @retval 无
23 */
24 void ILI9806G_SetPointPixel ( uint16_t usX, uint16_t usY )
25 {
26     if ( ( usX < LCD_X_LENGTH ) && ( usY < LCD_Y_LENGTH ) ) {
27         ILI9806G_SetCursor ( usX, usY );
28
29         ILI9806G_FillColor ( 1, CurrentTextColor );
30     }
31 }
```

ILI9806G_SetPointPixel 函数直接调用了 ILI9806G_SetCursor（实质上是 ILI9806G_OpenWindow 函数的封装）设置单个像素点的绘制窗口，然后调用 ILI9806G_FillColor 填充单个像素点，而像素点的颜色由全局变量 CurrentTextColor 表示。

利用这个 ILI9806G_SetPointPixel 函数，可以向液晶屏指定的 XY 坐标描绘单个像素点。

绘制矩形

类似地，使用 ILI9806G_OpenWindow 和 ILI9806G_FillColor 制作的绘制矩形操作见代码清单 28-12。

代码清单 28-12 绘制矩形

```
1 /**
2  * @brief  在 ILI9806G 显示器上画一个矩形
3  * @param  usX_Start : 在特定扫描方向下矩形的起始点 x 坐标
4  * @param  usY_Start : 在特定扫描方向下矩形的起始点 y 坐标
5  * @param  usWidth: 矩形的宽度（单位：像素）
```

```
6 * @param usHeight: 矩形的高度 (单位: 像素)
7 * @param ucFilled : 选择是否填充该矩形
8 *   该参数为以下值之一:
9 *       @arg 0 :空心矩形
10 *       @arg 1 :实心矩形
11 * @note 可使用 LCD_SetBackColor、LCD_SetTextColor、LCD_SetColors 函数设置颜色
12 * @retval 无
13 */
14 void ILI9806G_DrawRectangle ( uint16_t usX_Start, uint16_t usY_Start,
15                               uint16_t usWidth, uint16_t usHeight, uint8_t ucFilled )
16 {
17     if ( ucFilled ) {
18         ILI9806G_OpenWindow ( usX_Start, usY_Start, usWidth, usHeight );
19         ILI9806G_FillColor ( usWidth * usHeight ,CurrentTextColor);
20     } else {
21         ILI9806G_DrawLine ( usX_Start, usY_Start,
22                             usX_Start + usWidth - 1, usY_Start );
23         ILI9806G_DrawLine ( usX_Start, usY_Start + usHeight - 1,
24                             usX_Start + usWidth - 1, usY_Start + usHeight - 1 );
25         ILI9806G_DrawLine ( usX_Start, usY_Start, usX_Start,
26                             usY_Start + usHeight - 1 );
27         ILI9806G_DrawLine ( usX_Start + usWidth - 1, usY_Start,
28                             usX_Start + usWidth - 1, usY_Start + usHeight - 1 );
29     }
30 }
```

ILI9806G_DrawRectangle 函数分成两部分, 它根据输入参数 ucFilled 是否为真值决定绘制的是实心矩形还是只有边框的矩形。绘制实心矩形时, 直接使用 ILI9806G_OpenWindow 函数根据输入参数设置显示矩形窗口, 然后根据实心矩形的像素点个数调用 ILI9806G_FillColor 即可完成绘制; 而绘制空心矩形时, 实质上是绘制四条边框线, 它调用 ILI9806G_DrawLine 函数绘制, ILI9806G_DrawLine 函数的输入参数是用于表示直接的两个坐标点(x1,y1)与(x2,y2), 该函数内部根据数据关系, 使用这两个点确定一条直线, 最后调用 ILI9806G_SetPointPixel 函数一点一点地绘制成完整的直线。

关于 ILI9806G_DrawLine 画线函数、ILI9806G_DrawCircle 画圆函数等代码不再讲解, 它们都是根据数学关系在特点的位置显示坐标点而已。另外关于工程中的显示字符串的原理将在下一个章节详细说明。

设置液晶的扫描方向

控制液晶屏时, 还有一个非常重要的参数, 就是设置液晶屏的扫描方向, 见代码清单 28-13。

代码清单 28-13 设置液晶的扫描方向

```
1
2 #define          ILI9806G_LESS_PIXEL          480          //液晶屏较短方向的像素宽度
3 #define          ILI9806G_MORE_PIXEL          854          //液晶屏较长方向的像素宽度
4
5 //根据液晶扫描方向而变化的 xy 像素宽度
6 //调用 ILI9806G_GramScan 函数设置方向时会自动更改
7 uint16_t LCD_X_LENGTH = ILI9806G_LESS_PIXEL;
8 uint16_t LCD_Y_LENGTH = ILI9806G_MORE_PIXEL;
9
10 //液晶屏扫描模式, 本变量主要用于方便选择触摸屏的计算参数
11 //参数可选值为 0-7
12 //调用 ILI9806G_GramScan 函数设置方向时会自动更改
13 //LCD 刚初始化完成时会使用本默认值
14 uint8_t LCD_SCAN_MODE = 6;
```

```

15
16 /**
17 * @brief 设置 ILI9806G 的 GRAM 的扫描方向
18 * @param ucOption : 选择 GRAM 的扫描方向
19 * @arg 0-7 : 参数可选值为 0-7 这八个方向
20 *
21 * !!! 其中 0、3、5、6 模式适合从左至右显示文字，
22 *      不推荐使用其它模式显示文字 其它模式显示文字会有镜像效果
23 *
24 * 其中 0、2、4、6 模式的 X 方向像素为 240，Y 方向像素为 320
25 * 其中 1、3、5、7 模式下 X 方向像素为 320，Y 方向像素为 240
26 *
27 * 其中 6 模式为大部分液晶例程的默认显示方向
28 * 其中 3 模式为摄像头例程使用的方向
29 * 其中 0 模式为 BMP 图片显示例程使用的方向
30 *
31 * @retval 无
32 * @note 坐标图例：A 表示向上，V 表示向下，<表示向左，>表示向右
33 *      X 表示 X 轴，Y 表示 Y 轴
34
35 -----
36 模式 0:      .      模式 1:      .      模式 2:      .      模式 3:
37              A      .              A      .      A      .      A
38              |      .              |      .      |      .      |
39              Y      .              X      .      Y      .      X
40              0      .              1      .      2      .      3
41      <--- X0 o . <---Y1 o . o 2X---> . o 3Y--->
42 -----
43 模式 4:      .      模式 5:      .      模式 6:      .      模式 7:
44      <--- X4 o . <--- Y5 o . o 6X---> . o 7Y--->
45              4      .              5      .      6      .      7
46              Y      .              X      .      Y      .      X
47              |      .              |      .      |      .      |
48              V      .              V      .      V      .      V
49 -----
50
51                      LCD 屏示例
52                      |-----|
53                      | 野火 Logo |
54                      |-----|
55                      |
56                      |
57                      |
58                      |
59                      |
60                      |
61                      |-----|
62                      屏幕正面（宽 240，高 320）
63
64 *****/
65 void ILI9806G_GramScan ( uint8_t ucOption )
66 {
67     //参数检查，只可输入 0-7
68     if (ucOption > 7 )
69         return;
70
71     //根据模式更新 LCD_SCAN_MODE 的值，主要用于触摸屏选择计算参数
72     LCD_SCAN_MODE = ucOption;
73
74     //根据模式更新 XY 方向的像素宽度
75     if (ucOption%2 == 0) {
76         //0 2 4 6 模式下 X 方向像素宽度为 240，Y 方向为 320
77         LCD_X_LENGTH = ILI9806G_LESS_PIXEL;
78         LCD_Y_LENGTH = ILI9806G_MORE_PIXEL;
79     } else {

```



```

80      //1 3 5 7 模式下 x 方向像素宽度为 320, y 方向为 240
81      LCD_X_LENGTH = ILI9806G_MORE_PIXEL;
82      LCD_Y_LENGTH = ILI9806G_LESS_PIXEL;
83  }
84
85      //0x36 命令参数的高 3 位可用于设置 GRAM 扫描方向
86      ILI9806G_Write_Cmd ( 0x36 );
87  ILI9806G_Write_Data ( 0x08 | (ucOption<<5)); //根据 ucOption 的值设置 LCD 参数, 共 0-7 种模式
88      ILI9806G_Write_Cmd ( CMD_SetCoordinateX );
89      ILI9806G_Write_Data ( 0x00 );      /* x 起始坐标高 8 位 */
90      ILI9806G_Write_Data ( 0x00 );      /* x 起始坐标低 8 位 */
91      ILI9806G_Write_Data ( ((LCD_X_LENGTH-1)>>8)&0xFF ); /* x 结束坐标高 8 位 */
92      ILI9806G_Write_Data ( (LCD_X_LENGTH-1)&0xFF );      /* x 结束坐标低 8 位 */
93
94      ILI9806G_Write_Cmd ( CMD_SetCoordinateY );
95      ILI9806G_Write_Data ( 0x00 );      /* y 起始坐标高 8 位 */
96      ILI9806G_Write_Data ( 0x00 );      /* y 起始坐标低 8 位 */
97      ILI9806G_Write_Data ( ((LCD_Y_LENGTH-1)>>8)&0xFF ); /* y 结束坐标高 8 位 */
98      ILI9806G_Write_Data ( (LCD_Y_LENGTH-1)&0xFF );      /* y 结束坐标低 8 位 */
99
100     /* write gram start */
101     ILI9806G_Write_Cmd ( CMD_SetPixel );
102 }

```

当设置了液晶显示窗口, 再连续向液晶屏写入像素点时, 它会一个点一个点地往液晶屏的 X 方向填充, 填充完一行 X 方向的像素点后, 向 Y 方向下移一行, X 坐标回到起始位置, 再往 X 方向一个点一个点地填充, 如此循环直至填充完整个显示窗口。

而屏幕的坐标原点和 XY 方向都可以根据实际需要使用 0X36 命令来配置的, 该命令的说明见图 28-32。

36h	MADCTL (Memory Access Control)												
	D/CX	RDX	WRX	D17-8	D7	D6	D5	D4	D3	D2	D1	D0	HEX
Command	0	1	↑	XX	0	0	1	1	0	1	1	0	36h
Parameter	1	1	↑	XX	MY	MX	MV	ML	BGR	MH	0	0	00
	This command defines read/write scanning direction of frame memory.												
	This command makes no change on the other driver status.												
	Bit	Name			Description								
	MY	Row Address Order			These 3 bits control MCU to memory write/read direction.								
	MX	Column Address Order											
	MV	Row / Column Exchange											
	ML	Vertical Refresh Order			LCD vertical refresh direction control.								
	BGR	RGB-BGR Order			Color selector switch control (0=RGB color filter panel, 1=BGR color filter panel)								
MH	Horizontal Refresh ORDER			LCD horizontal refreshing direction control.									

图 28-32 液晶扫描模式命令 (摘自《ILI9806G 数据手册》)

0X36 命令参数中的 MY、MX、MV 这三个数据位用于配置扫描方向, 因此一共有 $2^3 = 8$ 种模式。ILI9806G_GramScan 函数就是根据输入的模式设置这三个数据位, 并且根据相应的模式更改 XY 方向的分辨率 LCD_X_LENGTH 和 LCD_Y_LENGTH, 使得其它函数可以利用这两个全局变量获屏幕实际的 XY 方向分辨率信息; 同时, 函数内还设置了全局变量 LCD_SCAN_MODE 的值用于记录当前的屏幕扫描模式, 这在后面计算触摸屏坐标的时候会使用到。设置完扫描方向后, 代码中还调用设置液晶显示窗口的命令

CMD_SetCoordinateX/Y (0X2A/0X2B 命令) 默认打开一个与屏幕大小一致的显示窗口，方便后续的显示操作。

调用 ILI9806G_GramScan 函数设置 0-7 模式时，各个模式的坐标原点及 XY 方向如图 28-33 所示。

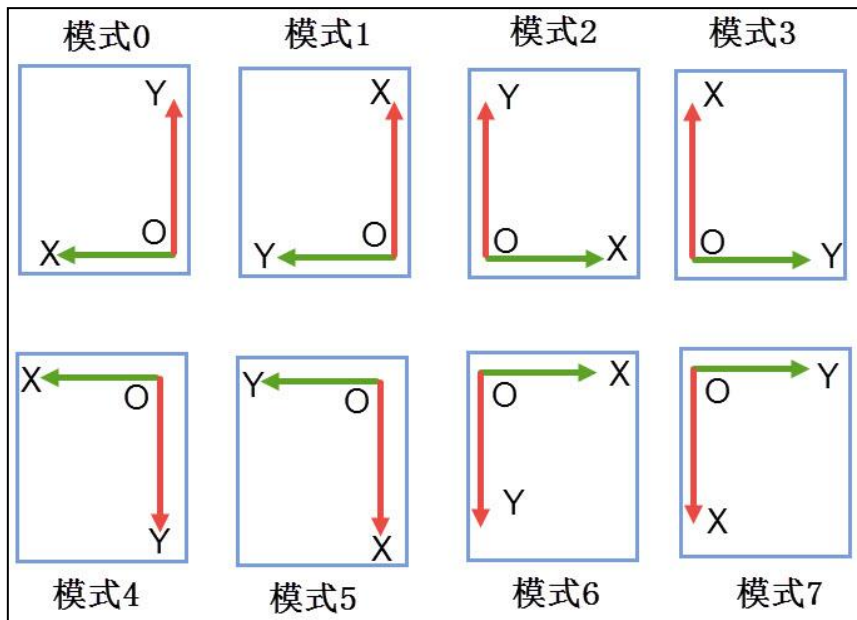


图 28-33 液晶屏的 8 种扫描模式

其中模式 6 最符合我们的阅读习惯，扫描方向与文字方向一致，都是从左到右，从上到下，所以本开发板中的大部分液晶程序都是默认使用模式 6。

其实模式 0、3、5、6 的液晶扫描方向都与文字方向一致，比较适合显示文字，只要适当旋转屏幕即可，使得用屏幕四个边沿作为正面看去都有适合的文字显示模式。而其它模式由于扫描方向与文字方向不一致，要想实现同样的效果非常麻烦，也没有实现的必要。

液晶屏全局初始化

利用前面介绍的各种函数，我们把它封装成 ILI9806G_Init 函数，见代码清单 28-14。

代码清单 28-14 液晶屏全局初始化函数

```
1 //液晶屏扫描模式，本变量主要用于方便选择触摸屏的计算参数
2 //参数可选值为 0-7
3 //调用 ILI9806G_GramScan 函数设置方向时会自动更改
4 //LCD 刚初始化完成时会使用本默认值
5 uint8_t LCD_SCAN_MODE = 6;
6
7 /**
8  * @brief ILI9806G 初始化函数，如果要用到 lcd，一定要调用这个函数
9  * @param 无
10  * @retval 无
11  */
12 void ILI9806G_Init ( void )
13 {
14     ILI9806G_GPIO_Config ();
15     ILI9806G_FSMC_Config ();
16
17     ILI9806G_Rst ();
18     ILI9806G_REG_Config ();
19 }
```

```
20 //设置默认扫描方向,其中 6 模式为大部分液晶例程的默认显示方向
21 ILI9806G_GramScan(LCD_SCAN_MODE);
22
23 ILI9806G_Clear(0,0,LCD_X_LENGTH,LCD_Y_LENGTH); /* 清屏,显示全黑 */
24 ILI9806G_BackLed_Control ( ENABLE );          //点亮 LCD 背光灯
25 }
```

本函数初始化 GPIO、FSMC 外设,复位液晶屏,然后向 ILI9806 写入配置参数,调用 ILI9806G_GramScan 函数设置默认的液晶扫描方向,并且写入基本的液晶屏配置,最后开启液晶屏的背光。在需要使用液晶屏的时候,直接调用本函数即可完成初始化。

3. 基本液晶显示例程的 main 函数

本章内容中配套了两个工程进行演示,它们的液晶驱动完全一样,仅是 main 函数里的应用层展示不同效果时稍有区别,先讲解基本的《液晶显示》例程,其 main 函数内容见代码清单 25-14。

代码清单 28-15 main 函数

```
1 /**
2  * @brief 主函数
3  * @param 无
4  * @retval 无
5  */
6 int main ( void )
7 {
8     ILI9806G_Init ();          //LCD 初始化
9
10    Debug_USART_Config();
11
12    printf("\r\n ***** 液晶屏英文显示程序***** \r\n");
13    printf("\r\n 本程序不支持中文,显示中文的程序请学习下一章 \r\n");
14
15    //其中 0、3、5、6 模式适合从左至右显示文字,
16    //不推荐使用其它模式显示文字 其它模式显示文字会有镜像效果
17    //其中 6 模式为大部分液晶例程的默认显示方向
18    ILI9806G_GramScan ( 6 );
19    while ( 1 ) {
20        LCD_Test();
21    }
22 }
```

程序中,调用了 ILI9806G_Init 函数初始化液晶屏,然后再初始化串口。(在实际测试中,若先初始化串口再初始化液晶屏,可能会导致错误,原因不明。所以在应用时,注意先初始化液晶屏再初始化串口)

初始化完成后,调用 LCD_Test 函数显示各种图形进行测试(如直线、矩形、圆形),见代码清单 28-16。具体内容请直接工程中阅读源码。LCD_Test 中还调用了文字显示函数,关于文字显示的原理在下一章再详细说明。

代码清单 28-16 液晶效果演示测试

```
1
2 /*用于测试各种液晶的函数*/
3 void LCD_Test(void)
4 {
5     /*演示显示变量*/
6     static uint8_t testCNT = 0;
7     char dispBuff[100];
```

```
8
9     testCNT++;
10
11     LCD_SetFont(&Font16x32);
12     LCD_SetColors(RED,BLACK);
13
14     ILI9806G_Clear(0,0,LCD_X_LENGTH,LCD_Y_LENGTH); /* 清屏，显示全黑 */
15     /*****显示字符串示例*****/
16     ILI9806G_DispStringLine_EN(LINE(0),"BH 4.5 inch LCD");
17     ILI9806G_DispStringLine_EN(LINE(2),"resolution:480x854px");
18     ILI9806G_DispStringLine_EN(LINE(3),"LCD driver:ILI9806G");
19     ILI9806G_DispStringLine_EN(LINE(4),"Touch driver:GT5688");
20
21     /*****显示变量示例*****/
22     LCD_SetTextColor(GREEN);
23
24     /*使用 c 标准库把变量转化成字符串*/
25     sprintf(dispBuff,"Count : %d ",testCNT);
26     ILI9806G_ClearLine(LINE(7)); /* 清除单行文字 */
27
28     /*然后显示该字符串即可，其它变量也是这样处理*/
29     ILI9806G_DispStringLine_EN(LINE(7),dispBuff);
30
31     /*****显示图形示例*****/
32     /* 画直线 */
33
34     ILI9806G_ClearLine(LINE(7));/* 清除单行文字 */
35     LCD_SetTextColor(BLUE);
36
37     ILI9806G_DispStringLine_EN(LINE(7),"Draw line:");
38
39     LCD_SetTextColor(RED);
40     ILI9806G_DrawLine(50,270,420,275);
41     ILI9806G_DrawLine(50,300,420,375);
42
43     LCD_SetTextColor(GREEN);
44     ILI9806G_DrawLine(50,370,420,475);
45     ILI9806G_DrawLine(50,400,420,475);
46
47     LCD_SetTextColor(BLUE);
48     ILI9806G_DrawLine(50,420,420,325);
49     ILI9806G_DrawLine(50,450,420,395);
50
51     Delay(0x2FFFFFFF);
52
53     ILI9806G_Clear(0,32*7,LCD_X_LENGTH,LCD_Y_LENGTH-32*7); /* 清屏，显示全黑 */
54     /*省略部分内容*/
55
56     Delay(0x2FFFFFFF);
57
58     ILI9806G_Clear(0,32*7,LCD_X_LENGTH,LCD_Y_LENGTH-32*7); /* 清屏，显示全黑 */
59
60 }
```

4. 液晶坐标方向演示的 main 函数

打开《液晶坐标方向演示》的例程，可看到其 main 函数与上一个工程有区别，见代码清单 28-17。

代码清单 28-17 液晶坐标方向演示例程的 main 函数

```
1 /**
2  * @brief 主函数
3  * @param 无
```

```
4  * @retval 无
5  */
6  int main ( void )
7  {
8      LED_GPIO_Config();
9
10     ILI9806G_Init ();           //LCD 初始化
11
12     Debug_USART_Config();
13
14     printf("\r\n ***** 液晶屏英文显示程序***** \r\n");
15     printf("\r\n 本程序不支持中文，显示中文的程序请学习下一章 \r\n");
16
17     while ( 1 ) {
18         //展示 LCD 的八种方向模式
19         LCD_Direction_Show();
20     }
21 }
22
23 /*用于展示 LCD 的八种方向模式*/
24 void LCD_Direction_Show(void)
25 {
26     uint8_t i = 0;
27     char dispBuff[100];
28
29     //轮流展示各个方向模式
30     for (i=0; i<8; i++) {
31         LCD_SetFont(&Font16x32);
32         LCD_SetColors(RED,BLACK);
33
34         ILI9806G_Clear(0,0,LCD_X_LENGTH,LCD_Y_LENGTH); /* 清屏，显示全黑 */
35
36         //其中 0、3、5、6 模式适合从左至右显示文字，
37         //不推荐使用其它模式显示文字 其它模式显示文字会有镜像效果
38         //其中 6 模式为大部分液晶例程的默认显示方向
39         ILI9806G_GramScan ( i );
40
41         sprintf(dispBuff,"o%d. X --->",i);
42         ILI9806G_DispStringLine_EN(LINE(0),dispBuff); //沿 X 方向显示文字
43
44         sprintf(dispBuff,"o%d.Y|V",i);
45         ILI9806G_DispString_EN_YDir(0,0,dispBuff); //沿 Y 方向显示文字
46
47         Delay(0x2FFFFFFF);
48
49         //显示测试
50         // * !!! 其中 0、3、5、6
51         //模式适合从左至右显示文字，不推荐使用其它模式显示文字
52         //其它模式显示文字会有镜像效果
53         LCD_Test();
54     }
55 }
```

本工程的 main 了函数中主要调用了 LCD_Direction_Show 函数，该函数主要添加了液晶屏在不同扫描模式下的显示效果演示，请直接观看程序的演示效果，了解液晶屏的各个扫描模式。注意：其中部分模式显示文字时会因为镜像效果导致无法阅读，这是由扫描模式决定的，并不是代码错误，只要使用适当的模式即可实现正常的文字显示效果。

下载验证

用 USB 线连接开发板，编译程序下载到实验板，并上电复位，液晶屏会显示各种内容。

